`ComplexGitSync` 0002.25

Documentation

Nicolas Flipo

September 2, 2026

Contents

Chapter 1

Getting Started

This chapter guides a new user through the Multi-repo Sync CLI. The documented entry point is `cgitsync`.

The primary lifecycle is:

1. `initialise(.cgs)` → clone all repositories → `READY`
2. `initialise(.gts)` → load or restore a saved snapshot → `READY`
3. `pull(.cgs|.gts)` → resynchronise an existing tree
4. `checkout` / `add` / `commit` / `push` / `tag` → tree-wide Git operations
5. `freeze` → write the next `.gts` snapshot and update the project-local `.lgr`

1.1 Prerequisites

- Python 3.11 or later.
- Git 2.30 or later available on `PATH`.
- Working Git authentication for every remote used by the workspace.

1.2 Installation

Install `ComplexGitSync` from source with the Pixi-managed environment:

```
$ git clone https://github.com/flipoyo/ComplexGitSync.git
$ cd ComplexGitSync
$ pixi install
$ pixi run cgitsync --help
```

1.3 Create or Check a Project Spec

A `.cgs` file is the local authoring spec for the repository tree. It normally contains only a project name and repository identifiers:

```
project = "CGSil1"
repos = [
  "gitlab:CGS_test/CGSil1",
  "gitlab:CGS_test/CGSil2",
  "github:flipoyo/CGSih1",
]
```

Each identifier uses `provider:owner/repository`. ComplexGitSync runs `PARSE -> NORMALIZE -> VALIDATE` and supplies deterministic defaults: `main` branches, `ssh`, `nested_config = auto`, and the repository name as its path. A unique repository matching the project name is mounted at `..` Advanced tables are reserved for overrides.

Canonical providers are `github`, `gitlab`, `codeberg`, and `custom`. Their known hosts are `github.com`, `gitlab.com`, and `codeberg.org`; SSH and HTTPS remotes are generated automatically. Custom hosting requires an explicit `gitprovider_url` and never guesses a host. For example, `codeberg:GX4G/GX4G` identifies owner `GX4G` and repository `GX4G` on Codeberg.

Copy `examples/template.cgs` when starting a specification. It combines plain repository identifiers with one inline override. Developers can compare it with `examples/normalized_template.cgs`, which shows the complete canonical document after normalization and is not intended as the usual hand-authored form.

The common inline options are `default_branch`, `fallback_branch`, `access_protocol`, `relative_path`, and `nested_config`. A relative path is resolved from the parent repository. Nested config can be `auto` to load the sole root-level `*.cgs`, `disabled` to stop discovery, or a relative `.cgs` path to select an exact file inside the repository.

```
$ pixi run cgitsync validate examples/complexgitsync.cgs
$ pixi run cgitsync view-tree examples/complexgitsync.cgs
```

A valid declared tree prints a lifecycle line such as:

```
DECLARED ready=false complete=true
```

1.4 Initialise the Workspace

Use `initialise` as the canonical first command. From a `.cgs` file it clones the full repository tree, validates it, writes the runtime snapshot under `<workspace>/.cgitsync/state/`, and leaves the tree `READY`.

```
$ pixi run cgitsync initialise examples/complexgitsync.cgs
```

The same canonical project definition can be authored directly. The `-repo` option is repeatable:

```
$ pixi run cgitsync initialise --project CGSil1 \
    --repo gitlab:CGS_test/CGSil1 \
    --repo codeberg:GX4G/GX4G
```

Use either a source file or `-project` plus `-repo`, never both. To serialize the CLI definition without initializing a workspace, use `create-cgs -project ... -repo ... -output FILE`.

When `-output-path` is omitted, `cgitsync` uses the workspace-level default `CGSPATH=../..`, then derives `CGSHOME=CGSPATH/<project-name>`. The explicit equivalent is:

```
$ pixi run cgitsync initialise examples/complexgitsync.cgs --output-path
    "$CGSPATH"
```

From a previously generated snapshot (resolved automatically via the project's `.lgr` register; pass the path explicitly only to override that discovery):

```
$ pixi run cgitsync initialise
```

Successful `.cgs` initialisation prints a readable operation sequence `GT-LOAD->GT-DISCOVER->GT-VALIDATE->` the explicit workflow `load->expand->validate->clone->gitignore`, the per-repository `git clone` action, the resolved root path, a report of any `.gitignore` changes, and a compact tree outline. Structured log records also put the operation code first, for example `GT-DISCOVER`,

GT-VALIDATE, FS-PURGE, GT-CLONE, or GT-GITIGNORE. If a stale partial checkout blocks the clone step, initialise fails explicitly and prints Try clean-init method. Use clean-init to insert a purge phase between validation and cloning:

```
$ pixi run cgitsync clean-init examples/complexgitsync.cgs
```

The explicit operation sequence is GT-LOAD->GT-DISCOVER->GT-VALIDATE->FS-PURGE->GT-CLONE->GT-GIT the matching workflow is load->expand->validate->purge->clone->gitignore. The purge phase can also be run on its own:

```
$ pixi run cgitsync purge examples/complexgitsync.cgs
```

purge removes generated clone state from CGSHOME: immediate child repository directories declared directly under the project root, and project *.lgr files. A persisted Git identity override at .cgitsync/master.toml (see the initialise entry in the Command Reference chapter for -git-user-name/-git-user-email) is workspace configuration, not clone state, and is left untouched.

1.5 Inspect the Runtime State

```
$ pixi run cgitsync view-tree
$ pixi run cgitsync view-tree --depth 2 --collapse parent_2
$ pixi run cgitsync status
```

If a command accepts an optional snapshot and none is provided, cgitsync discovers the latest .gts automatically via the project's .lgr register under CGSHOME/.cgitsync/. Each generated .gts snapshot actually lives under its own content-addressed CGSHOME/.cgitsync/state(<hash>)_<n>/ directory; pass an explicit path (e.g. -gts FILE, or a positional source, depending on the command) only to override that discovery. CGSHOME can be set explicitly; for initialise(.cgs) the default is CGSPATH=../.., and later commands can also discover the workspace by walking upward from the current directory.

1.6 Run the Git Cycle

Once the tree is READY, the Multi-repo Sync commands operate on every repository in deterministic order:

```
$ pixi run cgitsync pull
$ pixi run cgitsync checkout feature/my-branch
$ pixi run cgitsync add
$ pixi run cgitsync commit "feat: update project CGS#1"
$ pixi run cgitsync push
$ pixi run cgitsync freeze-release release-2026.05 "release 2026.05"
$ pixi run cgitsync freeze release-2026.05
$ pixi run cgitsync launch-release release-2026.05
```

pull walks the tree parent-first, including the project root: root -> parent -> leaf. Mutation commands such as add, commit, push, and freeze walk the opposite direction: leaf -> parent -> root. When local files block the safe pull, the CLI suggests cgitsync pull-force; that recovery command is destructive and discards local uncommitted/untracked work before force-updating the same tree.

Pass -gts <snapshot.gts> when you want to pin a command to an explicit snapshot, and pass -dry-run to add, commit, push, or freeze to preview the plan without mutating repositories.

1.7 Log Files

Every CLI command writes a timestamped log file. On Linux the default location is `/.local/state/ComplexGi` if `XDG_STATE_HOME` is set, logs are written under that state directory instead. The CLI prints the resolved path as `log_file=<path>`.

1.8 Next Steps

See Chapter 5 for the command reference, document formats, preflight checks, and troubleshooting notes.

Chapter 2

Direct Python Object API

This chapter shows how to manipulate `ComplexGitSync` objects directly (without the CLI wrapper), including loading a `.gts` snapshot, reconstructing the runtime registry, mirroring identity into `GitTree`/`GitRepo`, and propagating a tag target.

For normal Multi-repo Sync usage, prefer the CLI lifecycle documented in the README and Getting Started guide. The reference test topology is CGSil1 (`https://gitlab.com/CGS_test/CGSil1`): `initialise -> pull -> checkout/add/commit/push -> freeze -> launch_release`.

The lower-level object/API lifecycle remains: `load(.cgs) -> .gts LOADED -> expand(.cgs|.gts) -> .gts PENDING -> validate(.cgs|.gts) -> .gts READY -> pull(.cgs|.gts) -> checkout(.gts) -> add -> git(tree,"commit",msg) -> git(tree,"push") -> git(tree,"tag",name) -> freeze`.

2.1 Programmatic Project Configuration

Project configuration does not require the CLI or an existing `.cgs` file. The public facade accepts authoring values non-interactively and delegates normalization, static validation, and optional serialization to `cgs_format.py`. It performs no Git or network operations.

```
from ComplexGitSync import ComplexGitSyncClient

client = ComplexGitSyncClient()
document = client.configure(
    "GX4G",
    ["codeberg:GX4G/GX4G"],
    output_path="GX4G.cgs",  # optional
)
reference_tree = document.to_git_tree()
```

The CLI commands `configure`, `create-cgs`, and direct `initialise -project/-repo` authoring are adapters over this method.

2.2 From Empty Registry to READY via `.gts`

```
from pathlib import Path

from ComplexGitSync import GitRepo, GitTree, RefKind, TreeLifecycleState
from ComplexGitSync.orchestre import GtsDocument,
   build_registry_from_gts_document
from ComplexGitSync.git_tree import WorkingGitTree

# 1) Empty registry lifecycle
empty_registry = WorkingGitTree()
```

```
assert empty_registry.recompute_tree_state() == TreeLifecycleState.
    UNLOADED

# 2) Load the CGSil1 runtime snapshot (.gts) and rebuild the dependency
    registry
cgshome = Path("../..")
gts_doc = GtsDocument.from_toml(cgshome / ".cgitsync/state/CGSil1.gts")
registry = build_registry_from_gts_document(gts_doc)
assert registry.lifecycle_state == TreeLifecycleState.READY

# 3) Rebuild GitTree/GitRepo identity objects from discovered registry
    entries
tree = GitTree()
for entry in registry.values():
    tree.add_repo(
        GitRepo(
            project_owner_name=entry.project_owner_name or "owner",
            project_name=entry.project_name or entry.name,
            gitprovider=entry.gitprovider,
            access_protocol=entry.access_protocol,
            commit_sha=entry.commit_sha,
        )
    )
```

2.3 Object-Level Tag Propagation

```
# Set the same tag target across all registry entries (parent -> leaves)
tree.propagate_tag(registry, "v1.2.3")
for entry in registry.values():
    assert entry.target_ref_kind == RefKind.TAG
    assert entry.target_ref_name == "v1.2.3"
```

This object-level flow is the same state model used by the CLI and `ComplexGitSyncClient`, but exposed directly for advanced automation and tests.

2.4 READY Methods in the Client API

The same lifecycle model is exposed by `ComplexGitSyncClient`. In practice, the workflow is:

- `load(...)` is the step-1 name for direct source loading.
- `expand(...)` is the canonical step-2 name; it runs nested `.cgs` discovery from parents to leaves (recursive).
- `validate(...)` is the canonical step-3 name; it accepts both `.cgs` and `.gts` sources.
- `initialise(...)` bootstraps a tree and must finish in `READY`; `clone(...)` is the direct clone entry point for a `.cgs` source; `pull(...)` resynchronises an existing tree.
- `git(tree, command, *args)` is the unified step-5 interface for `"commit"`, `"push"`, and `"tag"` operations. Each follows leaf-first ordering.
- `freeze(...)` emits the next persisted `.gts` state.

```
from pathlib import Path

from ComplexGitSync import ComplexGitSyncClient

client = ComplexGitSyncClient()
cgspath = Path("../..")
cgshome = cgspath / "CGSil1"

# 1. load the CGSil1 test-case .cgs -> .gts LOADED
client.load("../CGSil1.cgs")

# 2. expand(.cgs/.gts) -> .gts PENDING
print(client.expand("../CGSil1.cgs"))

# 3. validate(.cgs/.gts) -> .gts READY
client.validate("../CGSil1.cgs")

# 4. clone(.cgs/.gts)
# Same default as the CLI: output_path defaults to CGSPATH=../..
client.initialise("../CGSil1.cgs")

# Equivalent explicit form:
client.initialise("../CGSil1.cgs", output_path=cgspath)

# Recovery path for partial/stale clone state:
client.clean_initialise_cgs("../CGSil1.cgs", output_path=cgspath)

# Cleanup only:
client.purge_cgs("../CGSil1.cgs", output_path=cgspath)

# 5. READY methods - unified git interface
registry = client.get_dependency_registry()
# Pull uses ROOT -> PARENT -> LEAF; every repository in the tree is a
    plain
# independent clone (there are no gitlinks) and receives its own git
    pull.
# A .gts source must be an actual snapshot path (find the current one
    via
# RuntimeStateStore.latest_snapshot_for(), the same lookup the CLI uses
    when
# a command's snapshot argument is omitted) -- unlike a .cgs source,
    there is
# no fixed, predictable filename to hard-code here.
client.pull("../CGSil1.cgs")
client.checkout("feature/my-branch")
# Mutations use LEAF -> PARENT -> ROOT.
client.add()
client.git(registry, "commit", "feat: update CGSil1 CGS#1")
client.git(registry, "push")
client.git(registry, "tag", "v1.2.3")

# 6. freeze -> next .gts id
client.freeze("release-2026.05")
```

2.5 Targeting a Single Path: `add` and `remove`

`add(paths=None)` and `remove(paths)` accept one or more filesystem paths (each relative to `CGSHOME`, or absolute) instead of operating on the whole tree:

```
# Stage only these two files, each in the repo that owns it -- every
# other repo in the tree is untouched.
client.add(paths=["notes.md", "deps/leaf/src/main.py"])

# Remove a single tracked file: deleted from disk, removal staged.
client.remove(["deps/leaf/obsolete.txt"])
```

Both resolve each path to its owning repo through the same helper, `resolve_repo_for_path(tree, path)` (`git_tree.py`) — the deepest (most specific) repo whose `absolute_path` contains the resolved path wins, so a nested child is correctly preferred over its parent. A path outside every repo in the tree raises `GitSyncError` immediately, before anything is staged or removed; for `remove`, a path that resolves to a directory or does not exist raises the same way rather than partially applying. `add()` with no `paths` keeps today's exact behaviour: every repo staged in full, leaf-first.

`remove` is a plain `git rm` (deletes from disk, stages the removal) — unrelated to `GitRunner.rm_cached` (index-only, used internally by `import_submodules` to drop a gitlink while keeping the child's working tree); the two are not interchangeable.

2.6 `.gitignore` Lifecycle Sync and Commit Identity

`initialise_cgs`, `initialise_cgs_document`, `clean_initialise_cgs`, `clean_init`, `restart`, and `pull` (for `.cgs` sources) all accept the same four optional keyword arguments, mirroring the CLI's `-commit-gitignore`, `-force-gitignore-sync`, `-git-user-name`, and `-git-user-email` flags:

```
from ComplexGitSync import ComplexGitSyncClient, MasterConfig

client = ComplexGitSyncClient()
client.initialise_cgs(
    "../CGSil1.cgs",
    output_path=cgspath,
    commit_gitignore=True,        # stage/commit/push each changed .
        gitignore
    force_gitignore_sync=False,  # pull-force fallback for a blocked
        repo
    git_user_name="cgitsync-bot",
    git_user_email="bot@example.com",
)

# The identity above is persisted to CGSHOME/.cgitsync/master.toml, so a
# later call on the same workspace picks it up without repeating it:
client.initialise_cgs("../CGSil1.cgs", output_path=cgspath,
    commit_gitignore=True)
assert MasterConfig.resolve_identity(cgshome, client.git_runner) == (
    "cgitsync-bot",
    "bot@example.com",
)
```

All four keyword arguments default to `False`/`None`: by default, every repository with children (root, or any nested repository that itself has further nested children) is safely pulled and has its `.gitignore` updated, but nothing is staged, committed, or pushed, and the commit identity — were a commit ever made — would be whatever `git config user.name`/`user.email`

already resolves to locally. `MasterConfig` (`master.py`) is the class behind the identity override: `configure()` sets it in memory for the current process, `persist(cgshome, ...)` writes it to `CGSHOME/.cgitsync/master.toml` (and also updates the in-memory value), `load(cgshome)` reads a previously persisted override back in, and `resolve_identity(repo_path, git_runner)` returns the `(user_name, user_email)` pair — or `(None, None)` to defer entirely to local git config. This file is workspace-local configuration, not part of the `.cgs`/`.gts` project spec, and is preserved (not deleted) by `purge`/`clean-init`.

2.7 Forcing a Clone Protocol

`initialise_cgs`, `initialise_cgs_document`, `clean_initialise_cgs`, `clean_init`, `bootstrap`, and `clone_cgs` additionally accept `force_access_protocol`, mirroring the CLI's `-force-protocol` flag on `initialise`, `bootstrap`, and `clean-init` (`restart`/`pull` do not accept it):

```
# Every repo this run clones -- root siblings and any child discovered
# later from a nested .cgs in a different repo -- is cloned over https,
# regardless of what its own .cgs entry says. No .cgs file is read
# differently or written.
client.initialise_cgs(
    "../CGSil1.cgs",
    output_path=cgspath,
    force_access_protocol="https",
)
```

`"ssh"` or `"https"`; `None` (the default) leaves every entry's own `.cgs`-declared `access_protocol` untouched. The override lives only in memory for the duration of the call – it is applied at the single point every clone's remote URL is built (`ComplexGitSyncClient._build_remote_url`), so it reaches root siblings and nested-discovered children alike without `cgs_format.py` parsing, or any `.cgs` file on disk, ever being touched differently.

2.8 Repository Discovery: `discover_repos`

`discover_repos(root_dir=None, *, max_depth=5, output=None)` scans a directory for git repositories already checked out on disk and drafts a `.cgs` from what it finds — the entry point for adopting a project that exists but has no specification yet:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()

# Read-only scan - nothing is cloned, fetched, or modified
report = client.discover_repos("/path/to/checkout")
for repo in report.repos:
    print(repo.relative_path, repo.identifier, repo.branch)

for warning in report.warnings:
    print("unresolved:", warning)

# Write the draft out (same call, with an output path)
report = client.discover_repos("/path/to/checkout", output="draft.cgs")
```

The method returns a `DiscoverReport` dataclass with fields `root` (the scanned directory), `repos` (a tuple of `DiscoveredRepo`), `cgs_entries` (authoring-form repo tables ready for `configure()`), `warnings`, and `project_name`. Each `DiscoveredRepo` carries `relative_path`, `absolute_path`, `remote_url`, `identifier`, `branch`, and `has_cgs`.

Repository addresses are resolved through the existing `parse_repo_id()` grammar; a repository with no `origin` or an unrecognised provider is reported in `warnings` and omitted from `cgs_entries` rather than guessed at. Passing `output` raises `GitSyncError` when nothing resolvable was found, so an empty draft is never written.

2.9 Submodule Migration: `import_submodules`

`import_submodules(repo_root, *, apply=False, output=None)` converts git submodules in a local repository to plain ComplexGitSync nested clones:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()

# Dry run - inspect without changing anything
report = client.import_submodules("/path/to/repo")
for sub in report.submodules:
    print(sub.name, sub.path, sub.url)

# Apply - convert and optionally emit a .cgs snippet
report = client.import_submodules(
    "/path/to/repo",
    apply=True,
    output="imported_submodules.cgs",
)
print(report.converted)   # tuple of converted submodule names
```

The method returns an `ImportSubmodulesReport` dataclass with fields `submodules` (all entries found), `applied` (whether the conversion ran), `converted` (names of converted submodules), and `cgs_entries` (authoring-form repo tables ready for `configure()`). When `apply=False` (the default), no files are modified. When `apply=True`, each submodule is removed from the index (`git rm -cached`), its stanza is removed from `.gitmodules`, and the parent's `.gitignore` is updated to ignore the child path. The staged changes should then be reviewed and committed manually.

2.10 Register Integrity: `verify`

`verify(cgshome, *, repair=False)` checks the hash-chained `.cgitsync/lgr` register under `cgshome` for tamper-evidence:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
report = client.verify("/path/to/CGSHOME")

if report.is_clean:
    print("register is clean")
else:
    for seq, finding, detail in report.findings:
        print(seq, finding.name, detail)

# Repair a stale HEAD cache (never rewrites or deletes an entry)
client.verify("/path/to/CGSHOME", repair=True)
```

Returns an `integrity.VerificationReport` (re-exported from the package root) whose `findings` list holds `(seq, Finding, detail)` tuples and whose `is_clean` property is `True`

when the list is empty. A register with no entries yet is reported clean — this is the common case today, since no command writes to this per-entry format yet (the CLI's `verify` command is the enforcement instrument; the migration of `SyncLedger`'s own writes to this format is a separate, future piece of work). `Finding` enumerates `BROKEN_LINK`, `BAD_ENTRY_HASH`, `SEQ_GAP`, `SEQ_DUPLICATE`, and `HEAD_STALE` for what this method actually checks (chain linkage and the `HEAD` cache); its remaining three members (`MISSING_STATE`, `ORPHAN_STATE`, `STATE_DIGEST_MISMATCH`) are reserved for a future store-level verification pass that cross-references entries against the actual `state(<hash>)_n/` directories on disk.

With `repair=True`, a stale `HEAD` cache is corrected in place. Register entries are never rewritten or deleted by this method, repair or not — a broken chain is reported, not silently healed.

Chapter 3

Architecture Overview

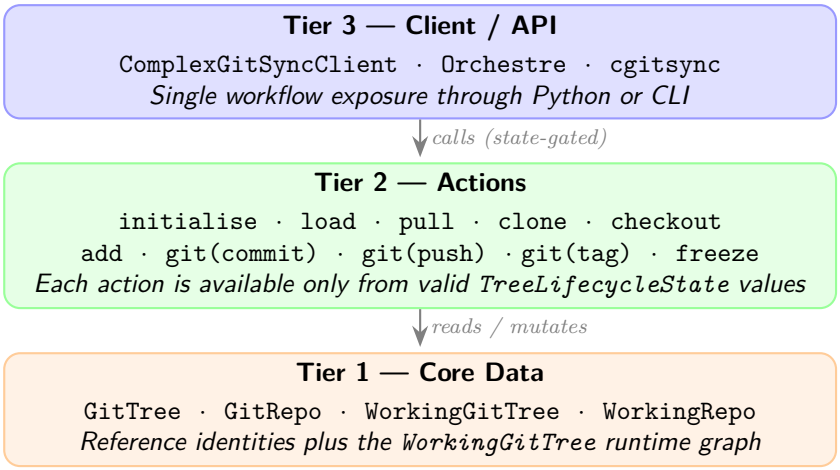
This chapter describes how `ComplexGitSync` is structured around three explicit tiers and explains the interaction between its classes. Related domain classes are grouped in modules according to the responsibility boundaries below.

3.1 Three-Tier Architecture

`ComplexGitSync` is organised into three tiers. Dependencies only flow **downward**: the Client calls Actions; Actions read and mutate Core Data; Core Data has no upward dependency.

Figure 3.1 shows the overall layout.

Figure 3.1: Three-tier architecture: Client/API exposes state-gated methods; Actions operate on Core Data; Core Data separates reference identities from the `WorkingGitTree` parent–leaf runtime graph.



3.2 Architectural Positioning

`ComplexGitSync` is positioned as a deterministic synchronisation layer *on top of Git*, not as a replacement for Git itself. It combines two distinct graph scopes:

- **Git DAG**: per-repository commit history and native remote semantics.
- **GitTree DAG**: workspace-level dependency graph coordinating parent/root/leaf propagation and ordering.

The system therefore differs from:

- plain Git usage (single repository scope),
- monorepo layouts (single storage boundary),
- submodule-only management (linking primitive without lifecycle orchestration),
- generic distributed sync engines (not constrained by Git-native '.gts'/'.lgr' deterministic contracts).

Figure 3.2 summarizes this positioning.

Figure 3.2: Architectural positioning matrix across Git scope and workspace coordination guarantees.

Approach	Primary scope	What it does not guarantee
Git (single repo)	Commit DAG per repository	Multi-repository propagation and coordinated lifecycle gating
Monorepo	One repository, one commit DAG	Independent repository identities and per-repo remote ownership
Submodule management	Parent repo references child revisions	Deterministic tree-wide mutation workflow and snapshot contracts
ComplexGitSync	Git DAG + GitTree DAG	Not a credential broker or remote policy authority

`freeze` materializes deterministic checkpoints as `.gts` snapshots (schema + SHA-256 hash), while `.lgr` assigns stable local ids and stores append-only ledger events for replayable local evolution.

Cross-declared nested repositories may initially resemble a local tangle at declaration time; runtime expansion normalizes this into a DAG via `fix_circularities`.

3.3 Tier 1 — Core Data

The reference model is centred on `GitTree`, which owns canonical `GitRepo` identities. Its runtime counterpart, `WorkingGitTree`, adds the parent–leaf graph of mutable `WorkingRepo` nodes. A project consists of one *root* repository that may nest *parent* repositories and terminal *leaf* repositories.

Figure 3.3 shows a representative project tree.

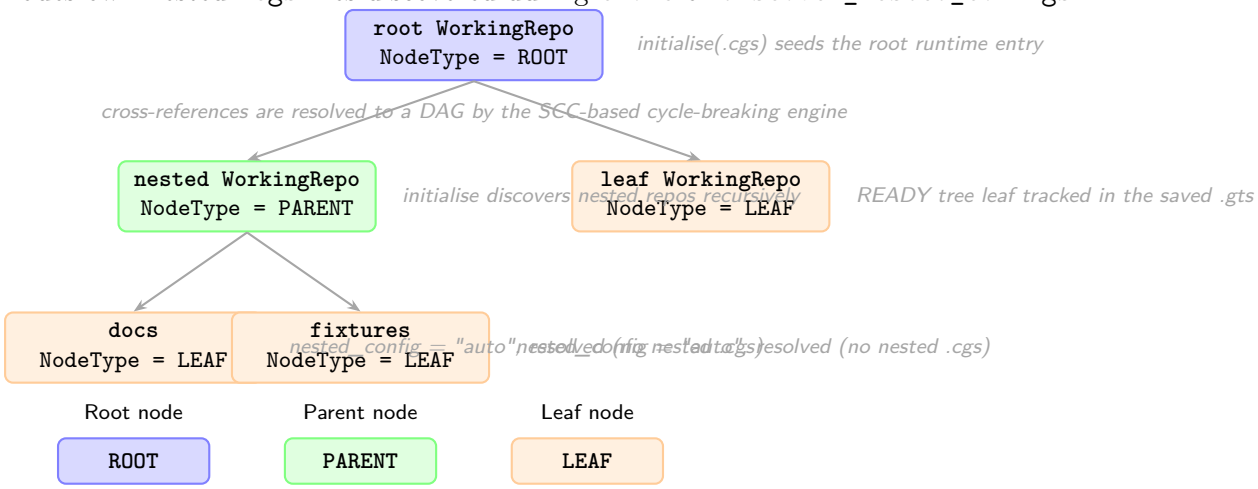
Registry entries in `WorkingGitTree` are indexed by a colon-separated repo ID derived from their position in the tree (e.g. `root:deps/child-repo:docs`). Cross-referenced parents therefore create a tangle-like declaration layer, but the expanded runtime graph remains a DAG after the cycle-breaking engine has run.

Cycle Breaking Engine

`fix_circularities` is a two-phase engine that converts a potentially cyclic dependency graph into a valid DAG:

1. **SCC detection (Tarjan's algorithm).** A directed dependency graph is built from the registry (edges: parent absolute path → child absolute path). `find_strongly_connected_components` identifies groups of paths that form a cycle. For each non-trivial SCC, an *Anchor* path is selected using three heuristics: (1) most external incoming edges, (2) closest to project root, (3) smallest SHA-256 hash. Back-edge entries — registry entries resolving to the anchor's path but parented inside a non-anchor SCC node — are flagged `is_external_reference = True` and removed.

Figure 3.3: `WorkingGitTree` is a parent–leaf graph of `WorkingRepo` nodes. Root and parent nodes own nested `.cgs` files discovered during `clone` or `discover_nested_configs`.



2. **Hash-compatibility deduplication.** Residual path-duplicate entries are removed when their synchronisation state is compatible with the canonical entry.

The clone engine uses a **sync stack** (set of absolute paths already entered into the clone pipeline) to exclude entries whose path is already being processed, providing defence-in-depth against any residual back-references that escape the registry cleanup phase.

`topological_sort(registry)` returns entries in parent-first order (Kahn's BFS algorithm) for safe sequential clone/pull operations. Runtime `pull` uses the same three-level direction: `ROOT -> PARENT -> LEAF`; every repository — root, parent, and leaf alike — is a plain independent clone and is pulled in that same parent-first order, rather than through any submodule linkage.

Figure 3.6 shows the class overview across all three tiers, `GitTree`/`GitRepo` among them; the full current module-by-module Ring-level dependency graph lives in `docs/DevGuide/architecture.md`.

3.4 Tier 2 — Actions

Actions are operations that read or mutate the Core tier. **Every action is gated by the current `TreeLifecycleState`**: actions that require a `READY` tree will refuse to execute and log a gating-refusal event if the tree is not ready.

Figure 3.7 shows the full lifecycle state machine. The public CLI presents this lifecycle through `initialise`, `pull`, and the READY-state sync commands; lower-level load/expand/-validate stages are internal steps of those workflows.

Action	Minimum state	Produces
`initialise(.cgs)`	none	clone tree + **READY**
`clean-init(.cgs)`	none	purge generated state + clone tree + **READY**
`purge(.cgs)`	none	removed generated root-level clone state
`initialise(.gts)`	none	loaded/restored **READY**
`pull(.cgs|.gts)`	READY	resynchronised **READY**
`checkout(.gts)`	READY	**READY**
`add`	**READY only**	READY
`commit <msg>`	**READY only**	READY
`push`	**READY only**	READY + updated hash
`tag <name>`	**READY only**	READY + updated tag
`freeze`	**READY only**	next `.gts` id

For `.cgs` refs, repositories can be declared by branch or by tag. Format validation is static and does not resolve either reference remotely. The runtime layer selects the declared target (tag takes precedence when both are present) and checks its availability through `GitRunner`.

The authoring boundary is explicitly `PARSE -> NORMALIZE -> VALIDATE`. TOML parsing produces authoring data; normalization expands `provider:owner/repository` identifiers and fills `main` branch defaults, `ssh`, automatic nested discovery, and relative paths. Validation receives only the complete canonical `CgsDocument`. Authoring syntax is therefore not the internal representation.

File and CLI authoring converge at this boundary. The CLI collects `-project` and repeatable `-repo` values, then calls the non-interactive `ComplexGitSyncClient.configure()` Python API. Python callers can use the same method directly. The facade delegates to `CgsDocument`; neither it nor the CLI owns repository grammar or normalization. Loading an equivalent `.cgs` file and using either definition path produce semantically identical canonical documents. `create-cgs` runs the same offline pipeline and serializes the result without entering Git runtime.

The boundary is also responsible for the reverse projection. `CgsDocument.to_git_tree()` creates the reference model and `CgsDocument.from_git_tree()` converts a reference or working tree back to canonical configuration. `GitTree.to_cgs()` remains only as a convenience delegate; it does not assemble repository tables or format TOML. The minimal serializer then removes deterministically reconstructible defaults. After the generated TOML is parsed and normalized again, its canonical representation must equal the one before serialization. Byte equality is not part of this contract.

Repository placement is parent-relative. At the top level the parent path is the project root; during nested discovery it is the repository whose `.cgs` is being expanded. Thus `../sibling` resolves to an existing sibling directory. Discovery compares normalized absolute paths and keeps an existing registry entry instead of inserting a duplicate.

Placement and traversal are independent. `nested_config = auto` searches the repository root for exactly one `*.cgs` — finding zero resolves the repository as a normal leaf, finding more than one raises immediately — `disabled` stops discovery through that reference, and a relative `.cgs` path selects an exact file inside the repository, failing if that named file does not exist. Explicit paths may not escape the repository root.

The textual `provider:owner/repository` grammar belongs exclusively to `cgs_format.py`. Its `parse_repo_id()` function owns syntax validation and splitting. Normalization uses that function, and CLI repository arguments must reuse it rather than introduce another parser.

The complete `.cgs` format pipeline is deterministic and offline. Parsing, normalization, validation, and serialization do not probe repositories or execute Git. Remote existence and branch/tag availability are resolved only after the validated document enters the runtime layer, through explicit `GitRunner` operations.

Provider behavior remains in `git_repo.py`: the `GitProvider` enum, canonical provider set, known-host map, and `RepoAddress` URL composer operate on already-separated identity fields. GitHub, GitLab, and Codeberg map respectively to `github.com`, `gitlab.com`, and `codeberg.org`. `custom` has no known-host entry and requires an explicit `gitprovider_url`; no fallback host is guessed.

Before any mutating workflow step (`commit`, `push`, `tag`, or `freeze`), the action layer now runs a workspace preflight validator. It checks dirty worktrees, detached HEADs, missing remotes, branch divergence, unresolved merges, and stale recorded `commit_sha` values. Diagnostics are severity-based: warnings report actionable-but-allowed states, while blocking errors stop the operation. `tag` still requires a clean worktree and a non-existing tag.

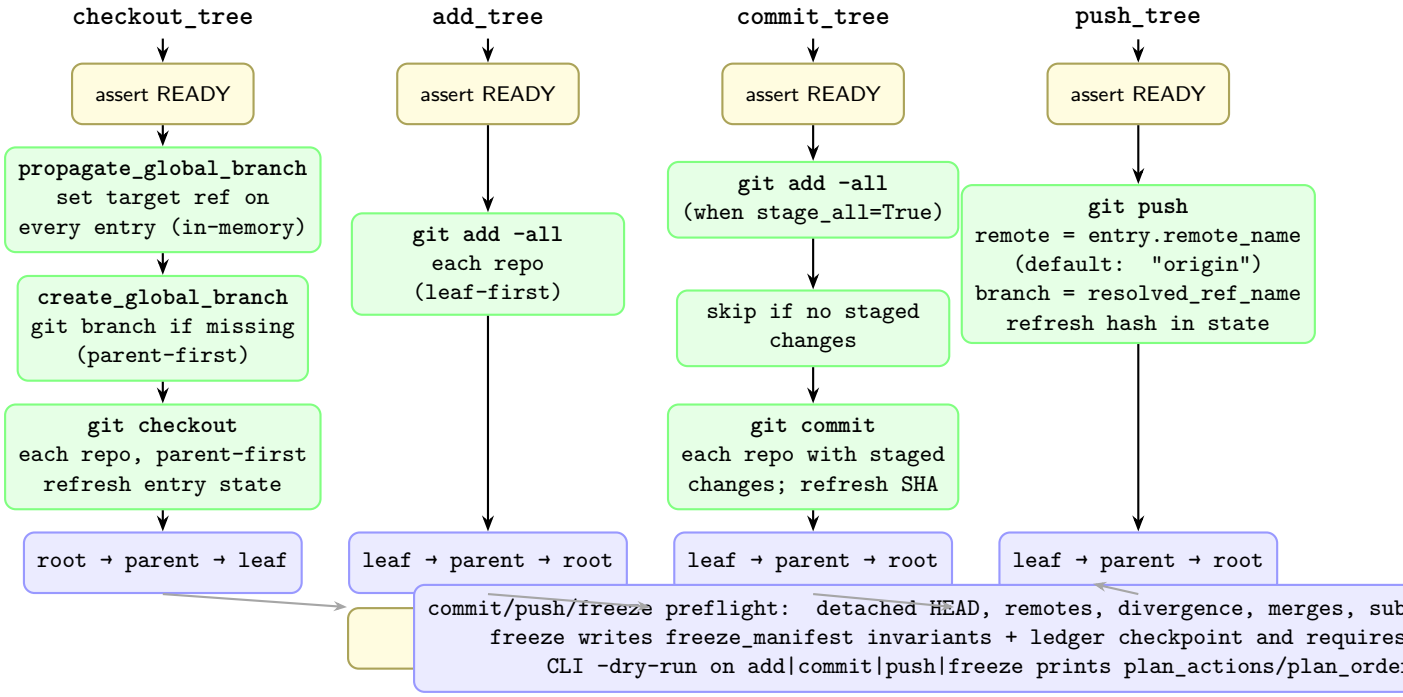
Formal `.gts` snapshot specification (T31)

`.gts` now carries explicit schema and deterministic hash metadata in the `[document]` table: `schema_version = "1.1"`, `hash_algorithm = "sha256"`, and `snapshot_hash = "<64-hex>"`.

The canonical digest is computed from a deterministic payload made of `project`, `tree_state`, and sorted `repo_state` records, excluding volatile generation metadata (`generated_at` and `command_origin`). Non-root entries require `parent_absolute_path`; READY repositories require `commit_sha`. This makes `.gts` a stable checkpoint primitive for repeatable workspace state comparison.

Figure 3.4 shows the internal sequence of the four tree-wide synchronisation actions implemented in `operations.py`.

Figure 3.4: Synchronisation operations: `pull/checkout_tree` process `ROOT -> PARENT -> LEAF`; `add_tree`, `commit_tree`, and `push_tree` iterate `LEAF -> PARENT -> ROOT` so that inner repos are synchronized before their parents.



3.5 Tier 3 — Client / API

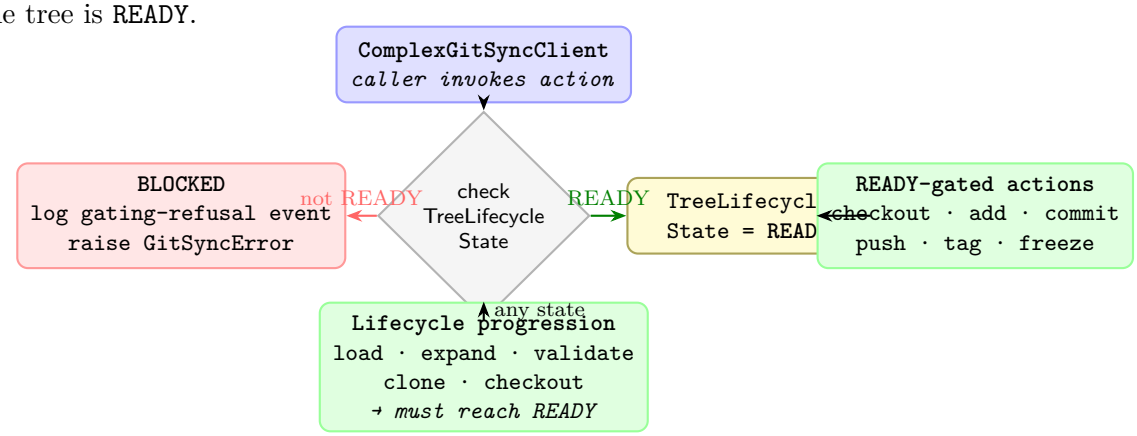
`ComplexGitSyncClient` is the single public facade. It:

- holds references to `Orchestre`, `GitRunner`, `RuntimeStateStore`, and the live `WorkingGitTree`;
- computes the current `TreeLifecycleState` on demand and gates every action against it;
- emits structured log events for every state transition, action start/end, fallback decision, and `.gts` write/load.

`Orchestre` is the coordination layer between the Client and the tree models. The CLI (`cli.py`) collects terminal arguments or interactive prompt values, then delegates to the reusable Client API. The Client delegates format semantics to `cgs_format.py`; runtime operations remain separate.

Figure 3.5 shows how the Client gates action access based on the live tree state.

Figure 3.5: Client state-gating: the Client reads `TreeLifecycleState` before delegating to an action. READY-gated actions (`checkout`, `add`, `commit`, `push`, `tag`, `freeze`) are blocked unless the tree is `READY`.



3.6 Module Layout by Tier

Listing 3.1: Source modules grouped by architectural tier

```
src/ComplexGitSync/
  # --- Tier 1: Core Data (centred on GitTree / GitRepo) ---
  git_repo.py                    GitRepo, WorkingRepo, RepoAddress,
                                 RepoNode, enums
  git_tree.py                    GitTree, WorkingGitTree,
                                 ProjectTreeState, tree utilities,
                                 sync_gitignore (.gitignore maintenance)

  # --- Tier 2: Actions ---
  operations.py                  propagate_global_branch,
     create_global_branch
                                 checkout_tree, commit_tree, push_tree
  orchestre.py                   discover_nested_configs(),
                                 build_registry_from_cgs_document, render
                                    helpers

  # --- Tier 3: Client / API ---
  orchestre.py                   Orchestre, ComplexGitSyncClient
  cli.py                         build_parser / main
  orchestre.py                   GitRunner, RuntimeStateStore,
                                 CommandRunLogger + create_run_logger
  master.py                      MasterConfig (workspace-local Git
     identity
                                 for ComplexGitSync-authored commits)

  # --- Document layer (cross-cutting) ---
  config_document.py             ConfigDocument (format-neutral base)
  cgs_format.py                  CgsDocument (.cgs parse/normalize/
     validate/serialize)
  orchestre.py                   GtsDocument
  errors.py                      exception hierarchy
```

3.7 Document Hierarchy

Figure 3.8 shows the document class hierarchy.

All persistent documents share the `ConfigDocument` base class, which provides format-agnostic serialisation (`to_toml`, `to_json`, `to_yaml`) and factory methods (`from_toml`, `from_json`, `from_yaml`). Subclasses add document-specific validation and convenience properties. The base is defined in `config_document.py`; `cgs_format.py` owns the complete `.cgs`-specific boundary, while `orchestre.py` consumes validated documents and retains runtime/orchestration responsibilities.

- **CgsDocument** (`.cgs`) — the authoring spec; describes the static project topology. *Never* a runtime snapshot.
- **GtsDocument** (`.gts`) — a generated snapshot capturing the exact state of the tree including commit SHAs and absolute paths. *Never* hand-edited.
- **Local Git Register** (`.lgr`) — the project-local register that assigns one stable local id to each generated `.gts` and tracks the current snapshot pointer.

3.8 Registry Model

Figure 3.9 shows the registry model.

`WorkingGitTree` is the authoritative in-memory graph. It maps *repo IDs* (colon-separated path strings such as `root:deps/child-repo`) to `WorkingRepo` instances. `WorkingRepo` holds both the static identity declared in `.cgs` and the dynamic state updated during operations.

Builder functions in `orchestre.py` translate document objects into a live working tree:

```
# Load from authoring spec
registry = build_registry_from_cgs_document(document, config_path)

# Load from snapshot
registry = build_registry_from_gts_document(snapshot_document)

# Serialise to snapshot
gts_doc = build_gts_document_from_registry(
    registry, command_origin="clone", source_cgs_path=path
)
```

Figure 3.6 is deliberately kept at Tier granularity rather than diagramming all of `src/ComplexGitSync/`'s current modules — see `docs/DevGuide/architecture.md` for the dev-facing companion that carries the full per-module Ring-level breakdown and dependency graph this chapter does not attempt to reproduce.

Figure 3.6: Class overview by tier. Headline classes for each of the three tiers, plus the cross-cutting Document layer and the `cli/` adapter package that the Tier model never described. Every module in `src/ComplexGitSync/` is named here, either as a class box or in a tier's module list; see `docs/DevGuide/architecture.md` for the full current module-by-module Ring-level dependency graph.

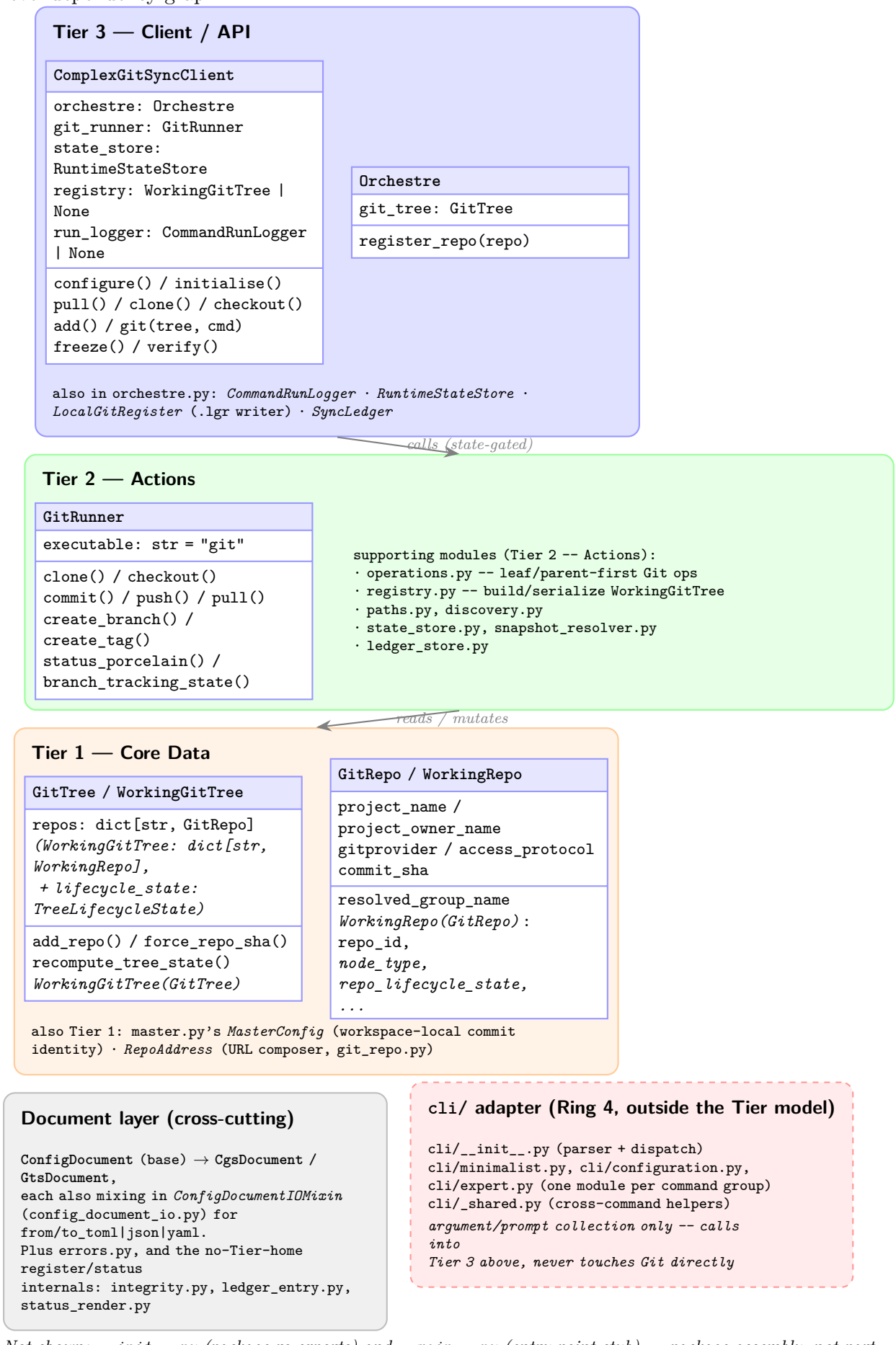


Figure 3.7: GitTree lifecycle state machine. The simplified user-facing lifecycle is `initialise(.cgs)` → clone → `READY`; `initialise(.gts)` → restore → `READY`; `pull(.cgs|.gts)` → resync → `READY`. Synchronized Git operations (`checkout`, `add`, `git("commit")`, `git("push")`, `git("tag")`, `freeze`) are gated on `READY`.

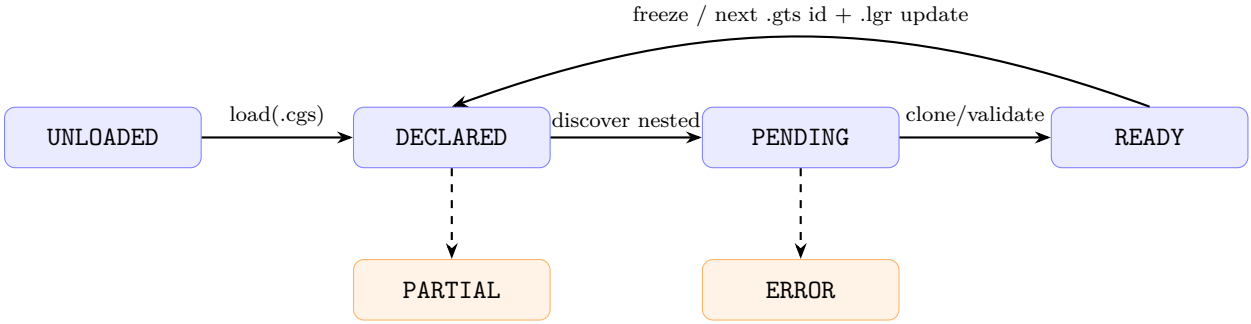


Figure 3.8: Document class hierarchy. `ConfigDocument` (`config_document.py`) is the abstract, I/O-free base; `ConfigDocumentIOMixin` (`config_document_io.py`) supplies the file-based `from/to_toml|json|yaml` methods. Each concrete document type combines both via ordinary multiple inheritance.

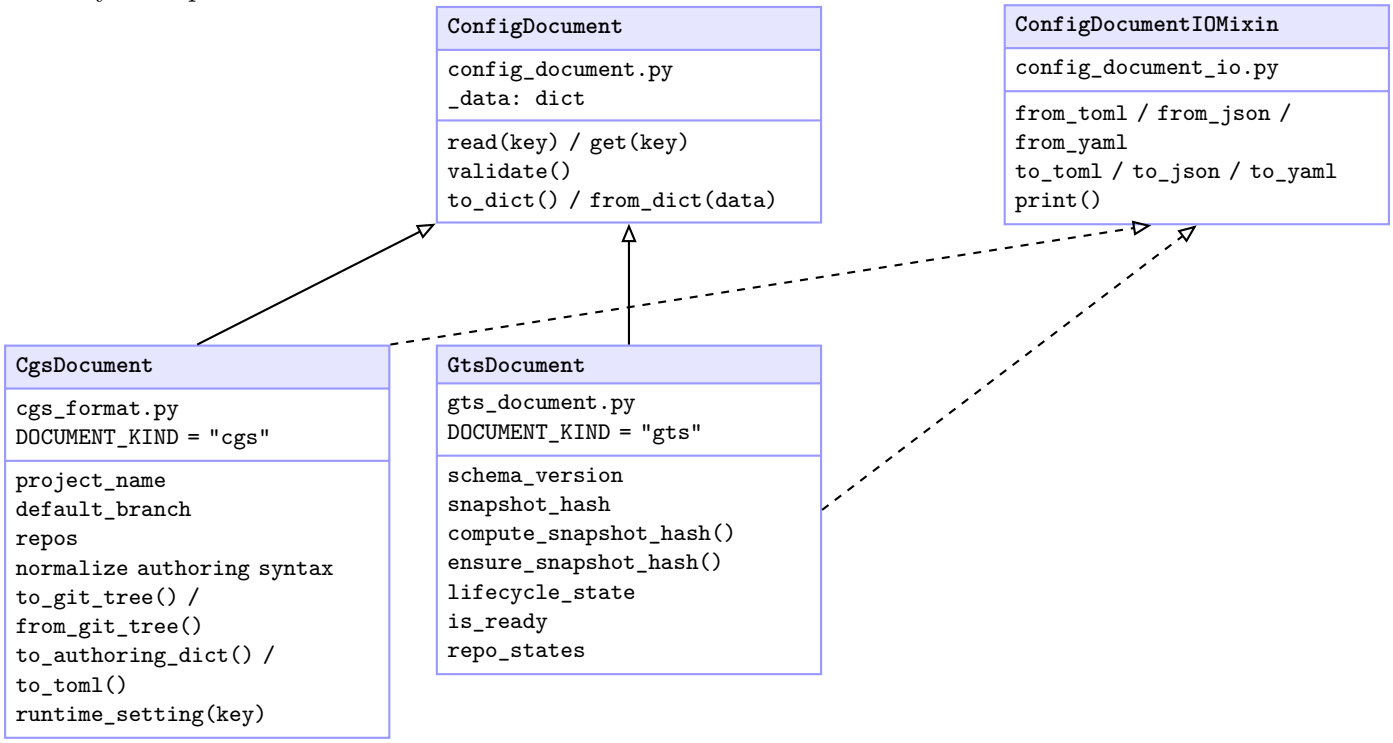
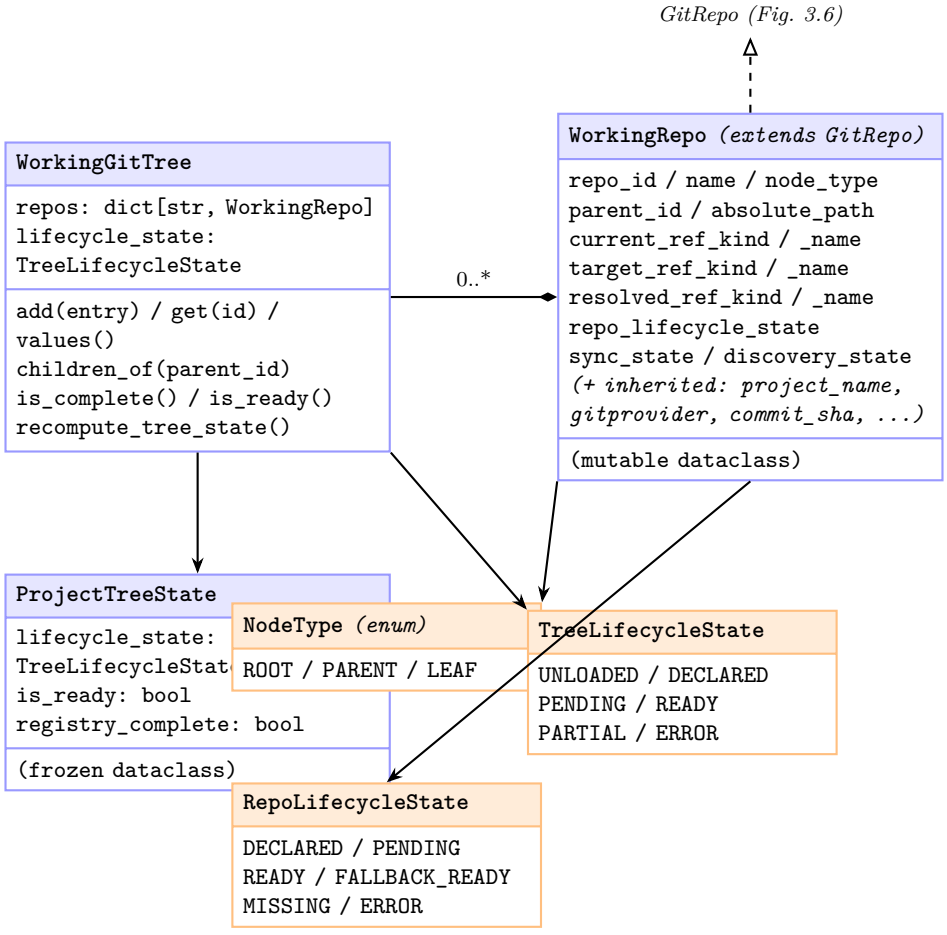


Figure 3.9: Registry model. `WorkingGitTree` is the authoritative in-memory graph; `WorkingRepo` *extends* `GitRepo` (Figure 3.6) rather than duplicating its identity fields, adding only the mutable tree-position and synchronisation state below.



Chapter 4

Writing ċgs Files: The Complete Authoring Guide

This section is the definitive reference for authoring `.cgs` (ComplexGitSync specification) files. It covers both the simplified shorthand syntax suitable for most use cases and the full advanced syntax for complex topologies, with practical guidelines, complete examples, and common pitfalls.

4.0.1 What is a ċgs File?

A `.cgs` file is a **TOML-formatted** specification that describes a tree of Git repositories to be managed as a single logical unit. It is the only hand-authored input in ComplexGitSync; all other documents (`.gts`, `.lgr`) are generated automatically at runtime.

Key properties:

- **Offline and deterministic:** Parsing, normalization, and validation require no network access. The same file always produces the same internal representation.
- **Never modified at runtime:** `.cgs` files are read-only during execution. Snapshots (`.gts`) capture workspace state; they do not update the specification.
- **Two equivalent forms:** A concise shorthand syntax for simple cases, and a verbose inline-table syntax for advanced overrides. Both normalize to the same canonical internal document.
- **Validation is strict:** Unknown fields, malformed identifiers, duplicates, and missing required keys are rejected before any Git operations begin.

4.0.2 Minimal ċgs: The Simplified Form

For the vast majority of projects, a `.cgs` file needs only two top-level keys: `project` (a string) and `repos` (a list of repository identifiers).

Basic Structure

```
project = "my-project"

repos = [
    "github:owner/repo1",
    "gitlab:group/repo2",
    "codeberg:user/repo3",
]
```

Repository Identifier Syntax

Each repository identifier follows the pattern:

```
provider:owner/repository
```

provider One of the supported Git providers: `github`, `gitlab`, `codeberg`, or `custom`.

owner The owner (user or organization/group) on the provider.

repository The repository name itself.

Nested namespace paths (e.g., GitLab subgroups) are supported. The final path segment is the repository name; all earlier segments join into `project_owner_name`:

```
repos = [
    "gitlab:my-group/my-subgroup/my-repo",  % 3-level namespace
    "github:my-org/my-repo",                % 2-level namespace
]
```

Valid providers and their default hosts:

Provider	Host	Remotes Generated
`github`	`github.com`	SSH and HTTPS
`gitlab`	`gitlab.com`	SSH and HTTPS
`codeberg`	`codeberg.org`	SSH and HTTPS
`custom`	explicit URL required	SSH and HTTPS

4.0.3 Normalization: What the Simplified Form Expands To

The simplified shorthand is automatically **normalized** into a complete canonical document. Every omitted field receives a deterministic default value.

Default Values Applied During Normalization

Field	Default	Applied To
`format_version`	`"1.0"`	document table
`default_branch`	`"main"`	project and each repo
`fallback_branch`	`"main"`	each repo
`access_protocol`	`"ssh"`	each repo
`nested_config`	`"auto"`	each repo
`relative_path`	repo name	each repo

Normalization Example

The following simplified input:

```
project = "demo"

repos = [
    "github:acme/core",
    "gitlab:acme/utils",
]
```

Normalizes to this canonical form (conceptually):

```
[document]
format_version = "1.0"

[project]
name = "demo"
default_branch = "main"

[[repos]]
gitprovider = "github"
project_owner_name = "acme"
project_name = "core"
repo_name = "core"
default_branch = "main"
fallback_branch = "main"
access_protocol = "ssh"
nested_config = "auto"
relative_path = "core"

[[repos]]
gitprovider = "gitlab"
project_owner_name = "acme"
project_name = "utils"
repo_name = "utils"
default_branch = "main"
fallback_branch = "main"
access_protocol = "ssh"
nested_config = "auto"
relative_path = "utils"
```

Important: The project's `default_branch` propagates to all repos unless explicitly overridden per-repo.

4.0.4 Inline Table Syntax: Overriding Defaults

When a repository needs non-default values for any field, use an **inline table** instead of a plain string. Only the fields that differ from defaults need to be specified.

Common Override Scenarios

```
project = "demo"

repos = [
    % Non-default branch
    { repository = "github:acme/core", default_branch = "develop" },

    % Different default and fallback branches
    { repository = "gitlab:acme/libs",
      default_branch = "next",
      fallback_branch = "main" },

    % Custom placement path
    { repository = "github:acme/docs",
      relative_path = "documentation" },

    % HTTPS access instead of SSH
    { repository = "codeberg:org/tool",
      access_protocol = "https" },
```

```
    % Disable nested config discovery
    { repository = "github:acme/legacy",
      nested_config = "disabled" },

    % Custom provider with explicit URL
    { repository = "custom:team/internal",
      gitprovider_url = "https://git.internal.company.com" },
]
```

Combined Overrides

Multiple overrides can be combined in a single inline table:

```
repos = [
    { repository = "github:acme/web",
      relative_path = "apps/web",
      default_branch = "production",
      fallback_branch = "staging",
      access_protocol = "https",
      nested_config = "disabled" },
]
```

4.0.5 Advanced Table Syntax: Legacy Form

The legacy advanced form uses explicit TOML tables. Fully supported but not recommended for new files.

```
[project]
name = "demo"
default_branch = "main"

[[repos]]
repository = "github:acme/core"

[[repos]]
repository = "gitlab:acme/utils"
default_branch = "develop"
relative_path = "lib/utils"
```

When to use advanced tables:

- Project-wide settings in the `[project]` table
- When you prefer visual separation between entries
- For compatibility with old ComplexGitSync versions

Note: The `[project]` table can also be inline: `project = { name = "demo", default_branch = "main" }`.

4.0.6 Repository Placement and Path Resolution

relative_path: Where Each Repository Lives

The `relative_path` field determines where a repository is cloned relative to its parent. Default: the repository name.

```
% Default: repo cloned at <parent>/repo_name
repos = ["github:acme/core"]

% Explicit: repo cloned at <parent>/apps/core
repos = [
    { repository = "github:acme/core", relative_path = "apps/core" }
]
```

Project Root Auto-Mount Rule

If exactly one repository name matches the project name, its `relative_path` is automatically set to `"."`:

```
project = "CGSil1"

repos = [
    "gitlab:CGS_test/CGSil1",  % Auto-mounted at "."
    "gitlab:CGS_test/CGSil2",  % At "./CGSil2"
    "github:flipoyo/CGSih1",   % At "./CGSih1"
]
```

Equivalent to:

```
project = "CGSil1"
repos = [
    { repository = "gitlab:CGS_test/CGSil1", relative_path = "." },
    { repository = "gitlab:CGS_test/CGSil2", relative_path = "CGSil2" },
    { repository = "github:flipoyo/CGSih1", relative_path = "CGSih1" },
]
```

Cross-Repository References

A repository can reference another repository in the same tree using relative paths:

```
% In CGSil2.cgs (nested under CGSil2/):
repos = [
    { repository = "github:flipoyo/CGSih1",
      relative_path = "../CGSih1" }
]
```

Here, `../CGSih1` resolves from `<CGSHOME>/CGSil2/` to `<CGSHOME>/CGSih1`, referencing the sibling declared elsewhere. No `nested_config` override is needed: discovery's absolute-path dedup guard already skips re-registering `CGSih1` once it is known under its canonical entry, before `"auto"` ever gets a chance to reopen its `.cgs`.

4.0.7 Nested Configuration Discovery

The `nested_config` field controls whether ComplexGitSync looks for additional `.cgs` files inside each repository.

`auto` (default) Loads the sole root-level `*.cgs` file if one exists; finding none is a normal leaf, not an error; multiple matches are rejected.

`disabled` Does not inspect the repository for nested config.

a relative path A path ending in `.cgs` loads that exact file; if the file doesn't exist, this is treated as an error, since an explicit path asserts that a specific file must be there.

Example: Multi-Level Nested Topology

```
project = "parent-project"

repos = [
    "github:org/parent",
    { repository = "github:org/child-a", nested_config = "auto" },
    { repository = "github:org/child-b", nested_config = "disabled" },
    { repository = "github:org/child-c", nested_config = "config/prod.
       cgs" },
]
```

4.0.8 Access Protocols and Custom Providers

SSH vs HTTPS

Default: SSH. To use HTTPS:

```
repos = [
    { repository = "github:owner/repo", access_protocol = "https" },
]
```

Generated remotes:

- SSH: `git@github.com:owner/repo.git`
- HTTPS: `https://github.com/owner/repo.git`

Custom Git Providers

For providers not in the built-in list, use `custom` with explicit URL:

```
repos = [
    { repository = "custom:team/project",
      gitprovider_url = "https://git.internal.mycompany.com" },
]
```

Important: `gitprovider_url` is **required** for `custom`.

```
% INCORRECT - will fail validation
repos = ["custom:team/project"]

% CORRECT
repos = [
    { repository = "custom:team/project",
      gitprovider_url = "https://git.internal.company.com" }
]
```

4.0.9 Branches: default_branch and fallback_branch

Branch Resolution Order

When cloning or pulling:

1. Repository's `default_branch`
2. Project's `default_branch`
3. Repository's `fallback_branch`
4. Project's `fallback_branch` (defaults to `main`)

Practical Branch Patterns

```
% All repos use "main" (default)
project = "my-project"
repos = ["github:owner/repo1", "github:owner/repo2"]

% All repos default to "develop"
project = { name = "my-project", default_branch = "develop" }
repos = ["github:owner/repo1", "github:owner/repo2"]

% Per-repo overrides
project = "my-project"
repos = [
    "github:owner/core",                    % Uses default "main"
    { repository = "github:owner/feature", default_branch = "dev" },
    { repository = "github:owner/stable",
      default_branch = "release", fallback_branch = "main" },
]
```

4.0.10 Complete Reference: All Recognized Fields

Top-Level Fields

Field	Type	Required	Default
`project`	string or table	yes	—
`repos`	list	yes	—
`document`	table	no	`{format_version = "1.0"}`

Project Table Fields

Field	Type	Required	Default
`name`	string	yes	—
`default_branch`	string	no	`"main"`

Repository Fields

Field	Type	Required	Default
`repository`	string	yes*	—
`gitprovider`	string	no	from `repository`
`project_owner_name`	string	no	from `repository`
`project_name`	string	no	from `repository`
`repo_name`	string	no	from `repository`
`default_branch`	string	no	from project or `"main"`
`fallback_branch`	string	no	`"main"`
`access_protocol`	string	no	`"ssh"`
`relative_path`	string	no	repo name (or "." if matches project)
`nested_config`	string	no	`"auto"`
`gitprovider_url`	string	no**	—

* Either `repository` (shorthand) or the combination of `gitprovider`, `project_owner_name`, `project_name` is required.

** `gitprovider_url` is required when `gitprovider = "custom"`.

4.0.11 Practical Examples

Example 1: Simple Monorepo-Style Project

```
project = "my-app"

repos = [
    "github:myorg/my-app",    % Root (auto-mounted at ".")
    "github:myorg/core",     % At ./core
    "github:myorg/ui",       % At ./ui
    "github:myorg/api",      % At ./api
]
```

Example 2: Mixed Providers with Custom Branches

```
project = "cross-provider"

repos = [
    "github:myorg/main-repo",
    { repository = "gitlab:mygroup/data", default_branch = "data-main"
       },
    { repository = "codeberg:user/tools", access_protocol = "https" },
]
```

Example 3: Complex Topology with Nested Configs

```
project = "big-system"

repos = [
    "github:org/root",
    { repository = "github:org/frontend",
      relative_path = "apps/frontend", nested_config = "auto" },
    { repository = "github:org/database",
      relative_path = "infra/db", nested_config = "disabled" },
    { repository = "github:thirdparty/lib",
      relative_path = "external/thirdparty-lib", nested_config = "
         disabled" },
]
```

Example 4: Real-World (cawaqsviz)

Based on `examples/cawaqsviz.cgs`:

```
project = { name = "CaWaQS-Viz", default_branch = "main" }

repos = [
    { repository = "gitlab:cawaqs/gviz/cawaqsviz",
      relative_path = ".", fallback_branch = "main" },
    { repository = "github:flipoyo/HydrologicalTwinAlphaSeries",
      relative_path = "external/HydrologicalTwinAlphaSeries",
      fallback_branch = "main" },
    { repository = "github:flipoyo/user_guide_CaWaQS-Viz",
      relative_path = "docs/CWV_user_guide",
      fallback_branch = "main" },
]
```

Neither child has a `.cgs` of its own, but no `nested_config` override is written for that: the default `"auto"` already resolves a repository with zero nested `*.cgs` matches as a normal leaf.

4.0.12 Authoring with CLI Tools

You do not need to write `.cgs` files by hand.

discover: Scan an Existing Checkout

```
$ pixi run cgitsync discover ~/work/myproject --write draft.cgs
$ pixi run cgitsync validate draft.cgs
```

configure: Interactive Prompt

```
$ pixi run cgitsync configure --output myproject.cgs
```

create-cgs: Direct CLI Authoring

```
$ pixi run cgitsync create-cgs \
    --project myproject \
    --repo github:owner/repo1 \
    --repo gitlab:group/repo2 \
    --output myproject.cgs
```

All three use the same offline pipeline as direct TOML parsing.

4.0.13 Validation and Common Errors

Always validate before use:

```
$ pixi run cgitsync validate myproject.cgs
```

Validation Checks

- Presence of required keys (`project`, `repos`)
- Valid repository identifier syntax
- Known provider names or custom with URL
- `custom` providers have `gitprovider_url`
- Valid `nested_config` values
- Valid `access_protocol` values
- No duplicate repository declarations
- Non-empty strings for all text fields

Common Errors and Fixes

`project is required` Add `project = "name"`.

`repos is required` Add `repos = [...]`.

`expected 'provider:owner/repository'` Fix identifier syntax.

`gitprovider_url is required for custom provider` Add `gitprovider_url = "https://..."`.

`duplicate repository` Remove one or make `relative_path` distinct.

`nested_config must be auto, disabled, or a path` Use valid value. Paths must end in `.cgs`.

4.0.14 Templates and Starting Points

Template Files

`examples/template.cgs` Copy for normal authoring.

`examples/normalized_template.cgs` Reference for canonical form.

Quick Start Checklist

1. Copy `examples/template.cgs` to `<your-project>.cgs`
2. Edit `project` name
3. Add repositories to `repos` list
4. Override defaults only when necessary
5. Validate: `pixi run cgitsync validate <file>.cgs`
6. Initialize: `pixi run cgitsync initialise <file>.cgs`

4.0.15 Best Practices

- **Start simple:** Use plain strings until you need overrides.
- **Use auto-mount:** Name one repo after project for automatic root.
- **Validate early and often:** After every edit.
- **Keep comments:** Document non-default values.
- **Avoid advanced tables:** Inline tables are more maintainable.
- **Use `nested_config = "disabled"` judiciously:** The default `"auto"` already resolves cleanly whether or not a repo has its own `.cgs` — including cross-references and cycle back-references, which the discovery dedup guard already prevents from re-registering. Reserve `"disabled"` for the rarer case where a repo does carry a `.cgs` you explicitly want ComplexGitSync to never open.
- **Prefer SSH:** Works well for most providers.
- **Be explicit with custom:** Always include `gitprovider_url`.

4.0.16 Internal Round-Trip Guarantee

ComplexGitSync guarantees a semantic round-trip:

$$\texttt{.cgs} \rightarrow \texttt{CgsDocument} \rightarrow \texttt{GitTree} \rightarrow \texttt{CgsDocument} \rightarrow \texttt{.cgs}$$

Parsing and normalizing a generated `.cgs` reproduces the canonical document. Whitespace and comments need not match byte-for-byte, but semantics must.

Chapter 5

User Guide

This chapter is the authoritative reference for every user-facing command, configuration option, document format, and error condition exposed by `ComplexGitSync`.

5.1 CLI Overview

The single entry-point is the `cgitsync` command.

`cgitsync` is a Pixi task, not a globally installed executable. This project is developed and run exclusively through Pixi (`pixi install` once, per checkout) — there is no `pip install` that puts a bare `cgitsync` on `$PATH`. Every invocation shown in this chapter as `pixi run cgitsync ...` must be typed exactly that way; a bare `cgitsync ...` will not be found by the shell.

```
$ pixi run cgitsync <command> [options]
```

Running `cgitsync` without a command prints a help summary and exits with code 0.

5.2 Lifecycle Contract

This guide follows the simplified canonical lifecycle:

1. `initialise(.cgs)` → clone all repos → `READY` *(new project)*
   `initialise(.gts)` → restore from snapshot → `READY` *(existing project)*
2. `pull(.cgs|.gts)` → resync an existing tree
3. `checkout(.gts)` / `add` / `commit` / `push` / `tag` → git operations on the tree
4. `freeze` → emit the next `.gts` id and update `<Project_name>.lgr`

The project-local `.lgr` register and sync ledger are both active (T24, T32). Every synchronisation operation is automatically recorded as an immutable DAG event in the `[[ledger]]` section of the `.lgr` file.

5.3 Commands

The CLI exposes three command groups once a `GitTree` is `READY`. The minimalist level is `initialise`, `bootstrap`, `clean-init`, `freeze-release`, `freeze-release-force`, `status`, `view-tree`, and `launch-release`. The expert level exposes the individual operations: `branch`, `checkout`, `purge`, `validate`, `clone`, `pull`, `pull-force`, `add`, `commit`, `push`, `tag`, `freeze`, `import-submodules`, and `verify`. The configuration level, `discover`, `configure` and `create-cgs`, authors a `.cgs`

specification without requiring existing state. Redundant inspection aliases such as `print`, `view-operation`, and `validate-topology` are not part of the public CLI surface.

5.3.1 `initialise`

```
$ pixi run cgitsync initialise <spec.cgs> [--output-path DIR] [--commit-
    gitignore]
    [--force-gitignore-sync] [--git-user-name NAME] [--git-user-email
       EMAIL]
    [--force-protocol {ssh,https}]
$ pixi run cgitsync initialise <snapshot.gts>
$ pixi run cgitsync initialise --project NAME --repo PROVIDER:OWNER/REPO
    [--repo ...] [--output-path DIR]
```

Unified initialisation entry point. Dispatches on file extension:

- `.cgs` source (clone mode): clones the full repository tree. The optional `-output-path` controls `CGSPATH`; `CGSHOME` is derived as `CGSPATH/<project-name>`. When omitted, `CGSPATH` defaults to `../...` On success, ends in `READY`. If clone setup fails because stale generated state is still present, the CLI prints `Try clean-init method`.
- `.gts` source (restore mode): loads a saved snapshot directly. On success, ends in the lifecycle state recorded in the snapshot.
- Direct CLI authoring: requires `-project` and at least one repeatable `-repo`. It is mutually exclusive with a source file. The CLI collects values only and calls `ComplexGitSyncClient.configure()`; that reusable Python API delegates parsing, normalization, and validation to `cgs_format.py` and produces the same canonical `CgsDocument` as an equivalent `.cgs` file.

All initialization paths end in a `READY` tree or fail explicitly.

Before a `.cgs` source is confirmed ready, every repository with children (root, or any nested repository that itself has further nested children) is safely pulled parent-first and has its `.gitignore` updated with the relative path of each immediate child — nested repositories are plain independent clones rather than gitlinks, so without this a parent's `git status`/`git add` would otherwise see a child's working tree as ordinary untracked content. This is logged as the `GT-GITIGNORE` phase, after `GT-CLONE`. If the safe pull for one of these repositories fails, `initialise` raises immediately and nothing is written; pass `-force-gitignore-sync` to instead fall back to a pull-force recovery for that one repository (never a force-push). By default nothing is staged, committed, or pushed by this step — it only writes the file and prints what changed. Pass `-commit-gitignore` to explicitly approve staging (`.gitignore` alone), committing (with a message listing exactly which children were added), and pushing each changed repository. The commit identity defaults to local git config; `-git-user-name`/`-git-user-email` override it and persist the override to `CGSHOME/.cgitsync/master.toml` via `MasterConfig`, so later invocations on the same workspace pick it up without repeating the flags.

`-force-protocol {ssh,https}` is an expert option, typically used systematically in CI rather than for everyday development. It overrides `access_protocol` in memory for every repository this run clones — including ones discovered later from a nested `.cgs` living in a different, separately-cloned repository — so no `.cgs` file is ever read differently or written to. Forcing `https` is the usual CI case: it removes the dependency on an SSH key or agent being present on the runner, a dependency that otherwise differs between a `push` run and a `pull_request` run (GitHub Actions withholds repository secrets from a pull request opened from a fork). Omitted (the default), each repository follows whatever protocol its own `.cgs` entry declares, exactly as without the flag.

5.3.2 bootstrap

```
$ pixi run cgitsync bootstrap <spec.cgs> <project_name> [--cgs-path DIR]
    [--force-protocol {ssh,https}]
```

Clones the full repository tree from a `.cgs` source into an isolated `CGSHOME`, for running `cgitsync` from its own standalone clone rather than nested inside the project tree it manages — one `ComplexGitSync` clone reused across many projects. It delegates to the same `ComplexGitSyncClient.clone_cgs` pipeline as `clone` below, and so shares its branch-fallback and nested-discovery behaviour, but differs from both `clone` and `initialise` in two ways:

- `project_name` is a required positional argument, not inferred from the `.cgs` document. It always forms the final path segment of `CGSHOME`, regardless of the specification's own `project` name.
- `CGSPATH` does not default to `../..` relative to the current directory (that default assumes `ComplexGitSync` is cloned inside the tree it manages — the nested-mode case `initialise` is for). When `-cgs-path` is omitted, `CGSPATH` instead defaults to a fresh `$HOME/.cgs/CGS<timestamp>/` directory, created automatically if it does not exist, so a bootstrapped project can never land inside `ComplexGitSync`'s own clone.

`-force-protocol {ssh,https}` behaves exactly as described under `initialise` above, applying here to every repository the run clones — including the root, which `bootstrap` (unlike `initialise`) also clones from scratch rather than assuming already checked out.

5.3.3 discover

```
$ pixi run cgitsync discover [ROOT] [--write FILE] [--max-depth N]
```

Scans `ROOT` (default: the current directory) for git repositories and drafts a `.cgs` from what is checked out, through the `ComplexGitSyncClient.discover_repos()` facade. This is the entry point for adopting a project that exists on disk but has no `.cgs` describing it yet.

The command is a pure read of the filesystem and of each repository's git config: nothing is cloned, fetched, staged, or modified, and no network call is made. Without `-write` it only prints what it found, matching the "report first, write only when asked" posture of `-commit-gitignore` and `import-submodules -apply`.

The walk descends up to `-max-depth` levels (default 5; `ROOT` itself is depth 0), treating every directory that contains a `.git` entry as a repository. It never descends *into* a `.git` directory: for a submodule the real git directory lives at `<parent>/.git/modules/<name>` while the child's own `.git` is a file, so walking into it would report the same repository twice.

For each repository, `origin`'s URL is read and converted to the canonical `provider:owner/repository` shorthand through the existing `parse_repo_id()` grammar — this command adds no second parser. `relative_path` is taken straight from the walk rather than from a repository name, so a child mounted at `external/Thing` is recorded there and not at `Thing`. `nested_config` is left unset on every entry: the default `auto` already resolves cleanly whether or not a repository carries its own `*.cgs`.

Repositories with no `origin`, or whose remote URL does not resolve to a registered provider, are reported as warnings and left out of the draft. They are never guessed at: a draft that silently invented an address would be worse than one that states what it could not determine. Always review the generated file, then `validate` it.

Only what is *checked out at scan time* can be found. A repository cloned without `-recurse-submodules` leaves its submodule paths as empty directories, and those are correctly not reported; recovering them from git metadata instead is `import-submodules`' job.

5.3.4 configure

```
$ pixi run cgitsync configure [--output FILE]
```

Interactively prompts for a project name, default branch, and one or more repositories (provider, owner, name, and per-repository overrides), then writes the resulting `.cgs` file through the same offline `ComplexGitSyncClient.configure()` facade used by `initialise -project/-repo` and `create-cgs`. The provider prompt accepts `github`, `gitlab`, `codeberg`, or `custom`; `custom` additionally prompts for the provider URL. When `-output` is omitted, the CLI prompts for a destination path, defaulting to `<project-name>.cgs`.

5.3.5 create-cgs

```
$ pixi run cgitsync create-cgs --project CGSil1 \
    --repo github:flipoyo/ComplexGitSync \
    --repo codeberg:GX4G/GX4G \
    --output CGSil1.cgs
```

Builds the canonical `CgsDocument` through the offline format pipeline and serializes concise `.cgs` TOML. It performs no Git or network work.

5.3.6 clean-init

```
$ pixi run cgitsync clean-init <spec.cgs> [--output-path DIR] [--commit-
   gitignore]
    [--force-gitignore-sync] [--git-user-name NAME] [--git-user-email
       EMAIL]
    [--force-protocol {ssh,https}]
```

Recovery-oriented initialisation for a `.cgs` source. It follows the same high-level pipeline as `initialise(.cgs)` (including the `.gitignore` lifecycle sync and `-force-protocol` behaviour described there), but inserts a cleanup phase before cloning:

`load->expand->validate->purge->clone->gitignore`

Use it when an earlier initialisation attempt left partial child checkouts. `CGSHOME/.cgitsync/master.toml` (the persisted Git identity override, if any) is preserved by the purge phase — it is workspace-local configuration, not generated clone state.

5.3.7 purge

```
$ pixi run cgitsync purge <spec.cgs> [--output-path DIR]
```

Removes generated clone state from the resolved `CGSHOME` without cloning. It deletes child repository directories declared directly under the project root and project-local `*.lgr` files. Nested directories that are not immediate root children, and `.cgitsync/master.toml` (see `clean-init` above), are left untouched by this command.

5.3.8 pull

```
$ pixi run cgitsync pull [spec.cgs|snapshot.gts] [--commit-gitignore]
    [--force-gitignore-sync] [--git-user-name NAME] [--git-user-email
       EMAIL]
```

Resynchronises the project tree. `.cgs` sources reload the source, discover nested configs, then resynchronise the loaded tree. `.gts` sources load the saved registry as the starting point and then run the same pull workflow. In both cases the operation starts at the project root: `root -> parent -> leaf`. Every repository — root, parent, and leaf alike — is a plain independent clone and receives its own `git pull -ff-only`. Use `launch-release` when you want to check out a frozen recorded release state instead of pulling current remotes. If local changes block this safe pull, the CLI prints `You can try cgitsync pull-force command`.

For a `.cgs` source, `pull` also runs the same `.gitignore` lifecycle sync described under `initialise` — including `-commit-gitignore`, `-force-gitignore-sync`, and the `-git-user-name`/`-git-user-` identity override — once the tree-wide pull completes. A `.gts` source runs no discovery, so there is nothing new for the sync to find. `pull-force` does not run this sync at all; it is a destructive recovery command, not a lifecycle path the sync is wired into.

5.3.9 pull-force

```
$ pixi run cgitsync pull-force [spec.cgs|snapshot.gts]
```

Destructively resynchronises the same tree. Every repository — root, parent, and leaf alike — is forced to the selected remote branch, parent-first, with `git fetch`, `git checkout -B <branch> FETCH_HEAD`, and `git clean -fd`. This command can discard local uncommitted changes and untracked files; it never force-pushes.

5.3.10 clone

```
$ pixi run cgitsync clone <spec.cgs> [--target-dir DIR] [--output-path
    DIR]
```

Clones the full repository tree declared by the `.cgs` file. This CLI command delegates to `ComplexGitSyncClient.clone`. When `-target-dir` is omitted, the project root is derived from the specification. `-output-path` provides the parent directory used to derive the project root.

For each repository, `cgitsync` tries the repo target branch first and falls back to the repo fallback branch when the preferred branch does not exist on the remote. Nested `.cgs` files are discovered after parent repositories are cloned, so explicit and automatic nested trees are materialised during the same command.

5.3.11 checkout

```
$ pixi run cgitsync checkout <branch> [--gts <snapshot.gts>] [--ref-kind
     branch|tag]
```

Checks out a branch or tag across the whole tree, parent-first. Loads the READY registry from `-gts`, then propagates the ref, creates missing local branches, and runs `git checkout` in every repo. On success the tree remains `READY` and a new `.gts` snapshot is written.

5.3.12 add

```
$ pixi run cgitsync add [PATH ...] [--gts <snapshot.gts>] [--dry-run]
```

With no `PATH` arguments (the default), stages all changes across the tree (`git add -all`), leaf-first. Requires `READY`; tree stays `READY`. With `-dry-run`, `cgitsync` prints the leaf-first execution plan and does not mutate repositories.

With one or more `PATH` arguments, each is resolved (relative to `CGSHOME`, or absolute) to the one repo in the tree that owns it and staged there individually (`git add - <path>`), leaving

every other repo untouched. A path outside every repo in the tree raises an error immediately, before anything is staged.

5.3.13 rm

```
$ pixi run cgitsync rm PATH [PATH ...] [--gts <snapshot.gts>] [--dry-run
    ]
```

Removes one or more tracked files, each resolved the same way as `add`'s targeted form: to the one repo in the tree that owns it, where it is deleted from disk and the removal staged (`git rm - <path>`). A plain tracked file only — a path that resolves to a directory, or that does not exist, raises an error rather than partially applying, and a path outside every repo in the tree raises before anything is removed. Requires `READY`; tree stays `READY`. With `-dry-run`, `cgitsync` prints the plan without mutating repositories.

Distinct from and unrelated to `import-submodules`' internal use of `git rm -cached` (index-only, dropping a gitlink while keeping the child's working tree) — `rm` deletes the file from disk.

5.3.14 commit

```
$ pixi run cgitsync commit <message> [--gts <snapshot.gts>] [--no-stage]
    [--dry-run]
```

Commits changes across the tree, leaf-first. Loads the READY registry from `-gts`. Repositories with no staged changes after the optional `git add -all` step (suppressed by `-no-stage`) are silently skipped. Requires a `READY` tree; the tree remains `READY` on success. Commit preflight now emits warnings for actionable workspace issues such as dirty worktrees, ahead branches, or stale recorded `commit_sha` values. With `-dry-run`, `cgitsync` prints the planned stage/commit sequence without performing git writes.

The commit message conventionally ends with `CGS#VERSION`.

5.3.15 push

```
$ pixi run cgitsync push [--gts <snapshot.gts>] [--dry-run]
```

Pushes all repositories to their configured remotes, leaf-first. Loads the READY registry from `-gts`. Each repo is pushed to `entry.remote_name` (defaulting to `"origin"`) on its currently resolved branch. Refreshes the stored hash in `GitTree`. Requires a `READY` tree; the tree remains `READY` on success. Push preflight blocks detached HEADs, missing remotes, unresolved merges, and behind/diverged branches; it warns when the local branch is ahead or when the loaded snapshot records an older `commit_sha`. With `-dry-run`, `cgitsync` prints the push plan without mutating any repository.

5.3.16 freeze-release

```
$ pixi run cgitsync freeze-release <name> <message> [--gts <snapshot.gts
    >] [--dry-run]
$ pixi run cgitsync freeze-release-force <name> <message> [--gts <
    snapshot.gts>] [--dry-run]
```

Minimalist release workflow from a READY tree. It chains public operations in order: `add -> commit -> pull -> push -> freeze`. The force variant uses `pull-force` instead of `pull`. The release name becomes the shared tag and freeze snapshot label; the message is used for the release commit.

The pull/pull-force step is skipped (not attempted) when the current branch has no upstream configured yet — e.g. a branch just created and checked out this same session, never pushed before. There is nothing to pull in that case; push (already upstream-aware, adding -u automatically the same way plain git push -u would) publishes the branch on its own. On any branch that already has an upstream, the workflow is unchanged.

5.3.17 freeze

```
$ pixi run cgitsync freeze <name> [--gts <snapshot.gts>] [--dry-run]
```

Performs a release freeze across all repos, leaf-first: stage/commit, create+push the shared release tag, refresh SHA/state, and write a .gts snapshot. Loads the READY registry from -gts. Requires READY and leaves the tree READY. Freeze preflight checks remote availability, branch alignment, detached HEADs, unresolved merges, and tag absence. Dirty worktrees and stale recorded commit_sha values are reported as warnings because freeze will stage, commit, and snapshot them. Freeze snapshots also include a deterministic freeze_manifest section that records freeze invariants (immutable snapshot, validated workspace, synchronized tag, release-name, ledger checkpoint) and declares launch_state as the canonical restore operation. Their immutable snapshot filename includes the release, for example gts-000001-release-2026.05.gts, while the ledger id remains gts-000001. With -dry-run, cgitsync prints the planned add/-commit/tag/push graph for the loaded workspace without mutating it.

5.3.18 import-submodules

```
$ pixi run cgitsync import-submodules REPO_ROOT [--apply] [--output FILE
    ]
```

Reads REPO_ROOT/.gitmodules and reports (or converts) every declared git submodule to a plain ComplexGitSync nested clone.

Without -apply (the default), the command performs a *dry run*: it prints each submodule's name, path, URL, and branch, and the .cgs entry that would be generated, without touching the repository.

With -apply:

- Verifies every submodule's working tree is clean (git status -porcelain must be empty); rejects dirty trees immediately.
- Runs git rm -cached <path> per submodule, dropping the gitlink from the index while leaving the child's working tree and .git directory intact (no re-clone, no local history lost).
- Removes the converted stanza from .gitmodules (deletes the file entirely when all stanzas are converted) and stages the change.
- Appends <path> to the parent's .gitignore using the same helper that the .gitignore lifecycle sync uses.

The -output FILE option (only effective with -apply) writes a validated .cgs snippet containing the converted submodule entries to FILE, suitable for integration into the project's main .cgs file.

After applying, review and commit the staged changes manually:

```
$ git -C REPO_ROOT status           # deleted .gitmodules, modified .
    gitignore
$ git -C REPO_ROOT commit -m "chore: retire git submodules"
```

Note: ComplexGitSync stores no credentials and has no private-repo authentication story. If a submodule's upstream is private and the ambient environment lacks access, the subsequent `cgitsync initialise` or `pull` will fail at the clone step. The `import-submodules` step itself (which only touches the local index and working tree) will still succeed.

5.3.19 `verify`

```
$ pixi run cgitsync verify [--search-dir DIR] [--repair]
```

Verifies the hash-chained `.cgitsync/lgr` register for tamper-evidence: every entry's `prev` field must match the previous entry's `entry_hash` (`BROKEN_LINK` if not), every entry's own `entry_hash` must match its recomputed canonical hash (`BAD_ENTRY_HASH` if not), `seq` numbers must be gap-free and unique (`SEQ_GAP`/`SEQ_DUPLICATE`), and the cached `HEAD` pointer must agree with the recomputed true head (`HEAD_STALE`). A register with no entries yet — the common case, since nothing currently writes to this format — is reported clean; that is not itself a finding.

`CGSHOME` is located the same way as `status`: from `-search-dir`, else `$CGSHOME`, else the current working directory's ancestors. Exits `0` when clean, `1` when any finding is reported.

With `-repair`, a stale `HEAD` cache is corrected to match the recomputed true head. **Entries themselves are never rewritten or deleted** by `verify`, repair or not — a broken chain is reported, not silently healed, since a register that can be edited back to looking clean provides no evidence of anything.

Scope note: this checks chain linkage and the `HEAD` cache only. Cross-referencing entries against the actual `state(<hash>)_n/` directories on disk (`MISSING_STATE`, `ORPHAN_STATE`, `STATE_DIGEST_MISMATCH`) is not implemented yet.

5.3.20 `launch-release`

```
$ pixi run cgitsync launch-release <name> [--gts <snapshot.gts>]
```

Checks out the frozen release tag `<name>` across the loaded READY tree, parent-first, and writes a fresh `.gts` snapshot. This is the release oriented CLI path for returning a workspace to a frozen version; direct tag creation remains available from the Python client API as `tag()`.

The integration suite validates local no-network lifecycle coverage through the CLI path, including initialisation and the READY git command cycle `add -> commit -> push -> freeze -> launch-release`.

5.3.21 `view-tree`

```
$ pixi run cgitsync view-tree [spec.cgs|snapshot.gts] [--depth N] [--
    collapse REPO]
```

Renders a deterministic terminal tree that focuses on repository topology and operational state: `name [branch] local_state sync_state`. Use `-depth` to limit visible levels and repeat `-collapse` to hide selected subtrees.

Branch Topology Propagation Rules (T35):

1. **Reference branch**: The root repository's current branch is the canonical reference for all repos.

2. **Leaf-to-root inheritance**: Branch targeting flows root-first via `propagate_global_branch`, verified on-disk by `ComplexGitSyncClient.validate_topology()` (Python API only) without issuing any git writes.

3. **Allowed divergence**: Repositories in a frozen (`TAG`) state produce a `tag_divergence` diagnostic that does *not* make the topology incoherent.

4. **Incoherent states** (blocking): `misaligned_branch` (different branch from root) and `detached_head` (detached HEAD without a tag reference).

5.3.22 `status`

```
$ pixi run cgitsync status [--gts FILE]
```

Summarises tree readiness and per-repository sync state. The table reports the local branch, upstream branch, local worktree state, ahead/behind tracking counts, current HEAD, and the commit recorded in the loaded `.gts`.

5.4 Document Formats

`.cgs` means **ComplexGitSync** specification, `.gts` means **GitTreeState** snapshot, and `.lgr` means **Local Git Register**.

5.4.1 Local Authoring Spec (`.cgs`)

A TOML file that describes the repository tree to manage. It is the only hand-authored input; it is never modified at runtime. The normal form has two top-level keys:

```
project = "CGSil1"
repos = [
  "gitlab:CGS_test/CGSil1",
  "gitlab:CGS_test/CGSil2",
  "github:flipoyo/CGSih1",
]
```

Repository identifiers have the form `provider:owner/repository`. Nested GitLab namespaces are supported; the final path segment is the repository name. The document pipeline is:

```
PARSE -> NORMALIZE -> VALIDATE -> canonical CgsDocument
```

The canonical providers and hosts are:

Provider	Host	Remote generation
`github`	`github.com`	SSH and HTTPS
`gitlab`	`gitlab.com`	SSH and HTTPS
`codeberg`	`codeberg.org`	SSH and HTTPS
`custom`	explicit URL required	SSH and HTTPS

Repository-ID parsing is provider-independent and deterministic. For example:

```
codeberg:GX4G/GX4G
```

normalizes to `gitprovider = codeberg`, `project_owner_name = GX4G`, and `repo_name = GX4G`. A custom identifier must use an inline table so the host is explicit:

```
{ repository = "custom:team/tool", gitprovider_url = "https://git.
   example.com" }
```

No host is inferred for `custom`. The configured access protocol then selects `git@host:owner/repository.gi` or `https://host/owner/repository.git`.

The textual grammar and its only parser, `parse_repo_id()`, live in `cgs_format.py`. The provider model in `git_repo.py` receives the already-separated provider, owner, and repository fields and composes remotes; it does not parse authoring shorthand.

The `.cgs` format pipeline is deterministic and offline. A syntactically valid repository may therefore reference a host, owner, branch, or tag that is not currently reachable. Remote existence and reference availability are checked only by explicit Git operations during runtime resolution and execution.

Normalization supplies `default_branch = main`, makes `fallback_branch` equal to that default, selects `access_protocol = ssh`, enables `nested_config = auto`, and uses the repository name as `relative_path`. If exactly one repository name matches `project`, its path is inferred as `.`.

Advanced inline tables are used only where a value differs from those defaults:

```
project = "demo"
repos = [
  "github:owner/demo",
  { repository = "github:owner/docs", relative_path = "documentation",
     access_protocol = "https" },
]
```

The legacy advanced `[project]` and `[[repos]]` tables remain accepted for project-wide branch settings, custom provider URLs, tags, and other explicit overrides. Unknown providers, malformed identifiers, duplicates, and missing `project` or `repos` are rejected before a `GitTree` is built.

Two checked-in files document this boundary. Copy `examples/template.cgs` for normal authoring. Its equivalent `examples/normalized_template.cgs` expands every inferred field and is provided as a developer reference for the canonical `CgsDocument` shape. The two files are regression-tested for semantic equivalence.

The repository also keeps four checked-in, didactically ordered tutorials under `docs/tutorials/` — easiest first, see `docs/tutorials/README.md` for the guided order — exercising this authoring surface end to end: `docs/tutorials/01_first_multi_repo_workspace.md` walks a fully exercised local sandbox topology (`CGSil1`) through the complete CLI lifecycle; `docs/tutorials/02_onboarding_a_` hand-authors a `.cgs` for the larger, real-world `cawaqs` tree and the point where `cgitsync` hands off to the project's existing `make`-based build workflow; `docs/tutorials/03_configuration_discovery_modes` covers the real-world `cawaqsviz` project (not synthetic) through two of the three ways `ComplexGitSync` can arrive at a working `.cgs` for a project it does not already control the source of (hand-authored, and `discover`); and `docs/tutorials/04_submodules_to_ready.md` covers the third way (`import-submodules`, for a project that already tracks its children as git submodules), then takes any of the three resulting `.cgs` files to a working `READY` tree.

`examples/cawaqsviz.cgs`, the hand-authored route from `docs/tutorials/03_configuration_discovery` above, describes `https://gitlab.com/cawaqs/gviz/cawaqsviz` and exercises several patterns above at once: a nested GitLab subgroup path (`cawaqs/gviz/cawaqsviz`, three segments rather than a plain `owner/repository` pair), an explicit `relative_path = "."` on the root entry rather than relying on the project-name auto-mount rule, two GitHub children mounted at non-default relative paths (`external/HydrologicalTwinAlphaSeries` and `docs/CWV_user_guide`). Neither child has a `.cgs` of its own, but no override is needed for that: the default `auto` nested discovery resolves a repository with zero `*.cgs` matches as a normal leaf. This file previously shipped with a nonexistent repository identifier and an invalid `default_branch`; it was corrected and given real end-to-end test coverage (`tests/integration/test_cgsi_topology.py`, and one verified run against the live repositories) as `AgentSpec/Onboarding_DevPlanTicket.md` Phase 1. See `docs/tutorials/03_configuration_discovery_modes.md` for the full worked ex-

ample, and `bootstrap` above for running `ComplexGitSync` against it without cloning `ComplexGitSync` inside the tree it manages.

Generated specifications use the same concise representation. The `GitTree.to_cgs()` convenience method delegates model projection to `cgs_format.py`; neither `GitTree` nor `GitRepo` formats TOML or implements the authoring grammar. Serialization omits every default that normalization can deterministically reconstruct and retains inline tables only for exceptional settings such as a non-default branch, path, protocol, tag, custom host, or discovery rule.

The supported round trip is semantic:

```
.cgs -> CgsDocument -> GitTree -> CgsDocument -> .cgs
```

Parsing and normalizing the generated file must reproduce the initial canonical document. Whitespace, comments, and TOML table layout need not be byte-for-byte identical.

5.4.2 Repository Placement and Nested Discovery

For every non-root entry, `relative_path` is relative to the parent repository represented by the current `.cgs`; it is not relative to the process working directory. At the top level that parent is the project root. While expanding a nested config, it is the repository containing that config. The default is the repository name. A unique repository matching the project name uses `.` because it represents the current root.

The `nested_config` option controls whether traversal continues after the repository is available:

- `auto` (default) loads the sole root-level `*.cgs` file, if one exists; finding none resolves the repository as a normal leaf; multiple matches are rejected as ambiguous.
- `disabled` does not inspect the repository for nested config.
- A relative path ending in `.cgs` loads that exact file inside the repository, failing if it does not exist. It may not escape the repository root.

For the integration topology, `CGSil2.cgs` contains:

```
{ repository = "github:flipoyo/CGSih1", relative_path = "../CGSih1" }
```

Because the config describes the tree below `CGSil2`, the path resolves from that directory:

```
<CGSHOME>/CGSil2/../CGSih1 -> <CGSHOME>/CGSih1
```

It therefore references the existing sibling declared by `CGSil1.cgs`, instead of cloning another `CGSih1` under `CGSil2`. Discovery recognizes the already-registered absolute path and keeps the canonical entry — this guard runs before `auto` would ever get a chance to reopen `CGSih1.cgs` through the cross-reference, so no `nested_config` override is needed here.

5.4.3 Git Tree State Snapshot (`.gts`)

A TOML file generated automatically by bootstrapping and release operations. It must never be hand-edited. It captures the exact commit SHAs and lifecycle state of the tree at a point in time.

Top-level sections:

- `[document]` — `format_version`, `schema_version`, `generated_at`, `command_origin`, `hash_algorithm`, `snapshot_hash`.
- `[project]` — project name and `root_absolute_path`.

- `[tree_state]` — `lifecycle_state`, `is_ready`, `registry_complete`.
- `[[repo_state]]` — one entry per repo; includes `commit_sha`, `sync_state`, and ref information.

Why both version fields and hash metadata exist:

- `format_version` identifies the broad document family and keeps long-range compatibility intent explicit.
- `schema_version` identifies the exact field-level contract used by validation and canonical snapshot serialization.
- `snapshot_hash` (with `hash_algorithm = "sha256"`) provides deterministic workspace identity: identical tree state produces identical digest even if volatile metadata (`generated_at`, `command_origin`) differs.

How this is used at runtime:

- During snapshot generation, ComplexGitSync writes `schema_version`, `hash_algorithm`, and canonical `snapshot_hash`.
- During load/validation, `snapshot_hash` is recomputed from the canonical payload and must match.
- The project-local `.lgr` register uses this canonical hash to deduplicate equivalent snapshots.

5.4.4 Local Git Register and Sync Ledger (`.lgr`)

A TOML file maintained per project at `<project-root>/<Project_name>.lgr`. It is updated automatically whenever a `.gts` snapshot is written.

Sections:

- `[register]` — current snapshot pointer (`current_snapshot_id`, `current_snapshot_hash`, `current_snapshot_path`).
- `[[snapshots]]` — list of stable `gts-XXXXXX` identifiers, one per unique workspace state, deduplicated by SHA-256 hash.
- `[[ledger]]` — append-only list of synchronisation events forming a DAG via `parent_sync_ids`.

Ledger event schema:

```
[[ledger]]
sync_id         = "lgr-000001"
parent_sync_ids = []
operation       = "clone"
timestamp       = "2026-05-20T19:48:50.159Z"
actor           = "username"
workspace_hash  = "5f7c8a2d..."   % 64-character SHA-256 hex digest
gts_snapshot_id = "gts-000001"
affected_repos  = ["demo", "dep-a"]
```

The `workspace_hash` field equals the canonical SHA-256 digest (`document.snapshot_hash`) of the associated `.gts` file, creating a direct link between each ledger event and the immutable workspace state it records. Events are parents-before-children in DAG order.

5.5 Worked Examples: From `CGSil1` to `cawaqsviz`

`ComplexGitSync` ships four checked-in `examples/` topologies. Levels 1 and 2 each have one matching tutorial under `docs/tutorials/`; Level 3's three routes span two more, since the third route (`import-submodules`) also covers reaching a `READY` tree (`docs/tutorials/README.md` lists all four tutorials in order). Run through the levels in order: each one adds exactly one new idea on top of the last, so together they form a guided path from the simplest possible `.cgs` to a real project with no `.cgs` of its own at all.

Every command below is a Pixi task. `cgitsync` is not a globally installed executable — always run `pixi run cgitsync ...`, never a bare `cgitsync ...` (see the §5 "CLI Overview" note above).

Level	Topology	Entry point	New idea introduced
1	`CGSil1`	hand-authored `.cgs`	minimal spec, name-matching auto-mount
2	`cawaqs`	hand-authored `.cgs`	the same minimal-field style as Level 1, at 19-repo scale
3a	`cawaqsviz`	hand-authored `.cgs`	explicit overrides on a real, irregular tree
3b	`cawaqsviz`	`discover`	drafting a `.cgs` from a checkout, no authoring at all
3c	`cawaqsviz`	`import-submodules`	migrating an existing git-submodules project

Levels 3a–3c are not three different projects: they are three different *starting points* for the same real project, `cawaqsviz`, chosen so this section can demonstrate every route `ComplexGitSync` offers for arriving at a working `.cgs` — write it by hand, derive it from a checkout, or migrate it from an existing submodules setup — without inventing a fourth toy topology to do it. It is the most advanced level not because `cawaqsviz` is a larger tree than `cawaqs` (it is much smaller, three repos against nineteen) but because, unlike either earlier level, it has no `.cgs` anywhere upstream to copy from.

5.5.1 Level 1 — `CGSil1`: the minimal synthetic topology

`CGSil1` is the easiest way into `ComplexGitSync`: a 4-repository, mixed-provider tree with every field left at its default. Its full `.cgs` and the rationale for each line are in §5 "Local Authoring Spec" above; `docs/tutorials/01_first_multi_repo_workspace.md` carries the full transcript with expected output for every step. What follows is the command sequence itself.

```
# 1. Fetch the root repo and cgitsync itself (nested mode: cgitsync is
#    cloned inside the tree it will manage).
$ git clone https://gitlab.com/CGS_test/CGSil1
$ cd CGSil1 && git clone https://github.com/flipoyo/ComplexGitSync
$ cd ComplexGitSync

# 2. Check the spec offline, no network beyond parsing.
$ pixi run cgitsync validate ../CGSil1.cgs

# 3. Preview the topology before touching disk.
$ pixi run cgitsync view-tree ../CGSil1.cgs

# 4. Clone every declared repo and reach READY.
$ pixi run cgitsync initialise ../CGSil1.cgs

# 5. The ordinary git cycle, run tree-wide from here on --- no path
```

```
#    argument needed, the .gts snapshot is discovered automatically.
$ pixi run cgitsync pull
$ pixi run cgitsync add
$ pixi run cgitsync commit "my commit message"
$ pixi run cgitsync push

# 6. Minimalist release: stage, commit, pull, push, freeze in one call.
$ pixi run cgitsync freeze-release v1.1.0 "release v1.1.0"

# 7. Return to a frozen release at any later point.
$ pixi run cgitsync launch-release v1.1.0
```

The only `.cgs` idea this level relies on is the **name-matching auto-mount**: because `CGSil1`'s repository identifier's last segment equals `project = "CGSil1"`, the root is inferred at `relative_path = "."` with no override written anywhere. That convenience is exactly what stops being safe once a project's identifier and display name diverge — which is where Level 3a picks up.

5.5.2 Level 2 — `cawaqs`: the same minimal style, at scale

`cawaqs` scales Level 1's minimal, no-override authoring style up to a real 19-repository build tree (root + 17 physics libraries across five GitLab groups + one required build-tooling repo), instead of a synthetic 4-repo one. Every entry's on-disk layout already matches the default `relative_path = <repo_name>`, so not a single `relative_path` override is written anywhere in `examples/cawaqs.cgs` — only the shared branch-fallback convention. Full narrative in `docs/tutorials/02_onboarding_a_real_build_tree.md`.

```
$ pixi run cgitsync validate examples/cawaqs.cgs
$ pixi run cgitsync bootstrap examples/cawaqs.cgs cawaqs-smoke-test
$ pixi run cgitsync view-tree --search-dir <path bootstrap printed>

# Move the whole 19-repo tree to a shared branch in one call, falling
# back to main per-repo wherever that branch does not exist.
$ pixi run cgitsync checkout my-feature-branch
```

Reaching `READY` here only replaces the *repository-fetching and branch-selection* half of `cawaqs`'s own `make_Cawaqs.sh`/`make_Cawaqs_from_branches.sh` scripts. `ComplexGitSync` does not compile anything: the remaining `make -f Makefile` build step is still the user's job, and only succeeds from this tree with `LIB_HYDROSYSTEM_PATH` left unset or pointed at the bootstrapped root — the alternative shared, decoupled install layout those scripts also support is out of scope for `ComplexGitSync`'s containment model.

`discover` (Level 3b below) is not a route into `cawaqs.cgs`: the 17 library repositories never coexist inside one directory tree until *after* a `.cgs` already lists them, so there is no single checkout for a filesystem walk to scan. Authoring this one from documented developer knowledge, the way Level 3a does for `cawaqsviz`, is the only available route.

5.5.3 Level 3 — `cawaqsviz`: three ways to the same `.cgs`

`cawaqsviz` (`https://gitlab.com/cawaqs/gviz/cawaqsviz`) is a real GitLab project, nested three path segments deep, with two GitHub children that were, until recently, plain git submodules. It has no `.cgs` of its own anywhere upstream. That makes it a good vehicle for showing all three ways `ComplexGitSync` can produce a working `.cgs` for a project it does not already control the source of, ordered here by how much the operator has to already know about the tree before running the command. Full narrative in `docs/tutorials/03_configuration_discovery_modes.md` (routes 3a–3b) and `docs/tutorials/04_submodules_to_ready.md` (route 3c).

3a — Write it by hand

The most explicit, and most error-prone, route: read the project's own `.gitmodules` and topology, then author `examples/cawaqsviz.cgs` directly. Its corrected content and the exact mistakes the first draft made (wrong nested-subgroup identifier, missing `relative_path` overrides) are already spelled out in §5 "Local Authoring Spec"; this level is about running it, not rederiving it:

```
$ pixi run cgitsync validate examples/cawaqsviz.cgs
$ pixi run cgitsync bootstrap examples/cawaqsviz.cgs cawaqsviz-demo
$ pixi run cgitsync view-tree --search-dir <path bootstrap printed>
```

`bootstrap` is used instead of `initialise` here because, unlike `CGSil1` above, this example is not meant to have `ComplexGitSync` cloned inside the tree it manages — see `bootstrap` in §5 for why the two commands pick different default locations.

3b — Derive it with `discover` (autodiscover mode)

The opposite route: start from nothing but a checkout, and let `ComplexGitSync` read the filesystem instead of writing the `.cgs` by hand. This is the route to prefer whenever a checkout is already available, since it cannot repeat the hand-authoring mistakes of 3a — `relative_path` falls out of the walk directly instead of being typed.

```
# A plain clone leaves submodule paths empty; initialise them first so
# discover has something to find at those paths. --init only (not
# --recursive): a nested submodule one level deeper would otherwise show
# up as a fourth repository discover has no use for here.
$ git clone https://gitlab.com/cawaqs/gviz/cawaqsviz cawaqsviz-scan
$ cd cawaqsviz-scan && git submodule update --init

# Dry run: report what discover sees, write nothing.
$ pixi run cgitsync discover . --max-depth 3

# Satisfied with the report: draft the .cgs.
$ pixi run cgitsync discover . --write cawaqsviz-discovered.cgs
$ pixi run cgitsync validate cawaqsviz-discovered.cgs
```

The draft reconstructs 3a's hand-corrected file field-for-field: the root at `relative_path = "."` and both children at their real submodule paths (`external/HydrologicalTwinAlphaSeries`, `docs/CWV_user_guide`) rather than their bare repo names. Neither child has a `.cgs` of its own, but `discover` writes no `nested_config` override for that: the default `"auto"` already resolves a repository with zero nested `*.cgs` matches as a normal leaf. See `discover` in §5 for exactly what is and is not reported (no network calls, unresolvable remotes are warned about and left out, never guessed at).

3c — Migrate it with `import-submodules`

The historical route: before either `.cgs` above existed, `cawaqsviz` tracked both children as real git submodules. This mode starts from that state and converts it, rather than describing a tree that is assumed to already be plain clones.

```
# Dry run against a real cawaqsviz checkout that still has submodules:
# reports name, path, URL, branch per submodule; changes nothing.
$ pixi run cgitsync import-submodules /path/to/cawaqsviz

# Convert: drop the gitlinks, retire .gitmodules, extend .gitignore,
# and emit the equivalent .cgs entries.
$ pixi run cgitsync import-submodules /path/to/cawaqsviz \
```

```
    --apply --output cawaqsviz_submodules.cgs

# Review and commit the resulting working-tree changes as usual.
$ cd /path/to/cawaqsviz && git status
$ git commit -m "chore: retire git submodules in favour of
   ComplexGitSync"
```

The full before/after picture (index, `.gitmodules`, `.gitignore`) and the automated test that proves it against fixture repositories are in `docs/tutorials/04_submodules_to_ready.md` §1 — this level is the condensed command sequence, not a restatement of that walkthrough. `import-submodules` and `discover` answer different questions about the same tree: `discover` reports what is *checked out*, `import-submodules` reads what git *declares* in `.gitmodules` and acts on it — a submodule path that was never initialised is invisible to 3b but not to 3c.

A shared caveat across all three

`ComplexGitSync` stores no credentials and has no private-repository authentication story (§5 "`import-submodules`" and "`discover`"). All three routes above assume every repository in the tree is reachable anonymously or through credentials the ambient environment (`ssh-agent`, an HTTPS credential helper) already holds.

5.6 Configuration and Environment

`ComplexGitSync` uses project files for topology and does not require global topology configuration. Snapshot discovery can use `CGSHOME`; when it is not set, the `initialise(.cgs)` output-path default is `CGSPATH=../..` (nested mode: `ComplexGitSync` cloned inside the tree it manages); later commands can also walk upward from the current directory to find `.cgitsync/state/`. `bootstrap(.cgs)` uses a different default instead — `CGSPATH=$HOME/.cgs/CGS<timestamp>/`, created automatically — since it is meant for a `ComplexGitSync` clone that is not nested inside any project tree; see `bootstrap` above. The logging and runtime-state subsystems use the user state directory by default:

- logs: `/.local/state/ComplexGitSync/logs/`
- latest runtime-state mapping: `/.local/state/ComplexGitSync/runtime/`

If `XDG_STATE_HOME` is set, these directories are rooted there instead.

5.7 Error Reference

`ConfigValidationError` Raised when `project` or `repos` is missing, a repository identifier is malformed, or a canonical field is invalid (for example an unknown `gitprovider`).

`GitSyncError` Raised when clone-time Git operations fail, for example because the remote branch cannot be resolved or the target directory is already occupied.

Exit code 1 General CLI failure (parse error, validation error).

Exit code 2 Command exists but is not yet implemented.

5.8 Troubleshooting

"`gitprovider_url` is required for custom provider" Add a `gitprovider_url` field to the offending `[[repos]]` entry in your `.cgs` file.

`validate` exits non-zero despite a correct-looking file Check that `project` is present and every `repos` entry uses `provider:owner/repository`. For advanced `[[repos]]` blocks without a `repository` shorthand, include `project_owner_name` and `project_name`.

`Clone destination already exists and is not empty` Re-run `clone` with `-target-dir` to force a clean location, or let `cgitsync` choose a suffixed default directory.

`No cloneable branch found for <repo>` Confirm that the remote exposes either the repo's `default_branch` or its `fallback_branch`.

PyYAML import error when calling `from_yaml/to_yaml` Add PyYAML in the Pixi environment: `pixi add pyyaml`.